
Kalshi Trading Platform

Technical Whitepaper

Gabriel Kagan

May 2026

Contents

Part I: Introduction	4
1.1 Scope of this document	4
1.2 Operational state at time of writing	4
1.3 First-principles statement of the system	5
Part II: System Architecture	5
2.1 Overview	5
2.2 Source tree	6
2.3 Runtime topology	8
2.4 Data flow	9
Part III: Probability and Calibration Pipeline	9
3.1 The four-stage probability cascade	9
3.2 Layer 0 — Raw statistical probability	10
3.2.1 Z-score construction	10
3.2.2 NIG CDF	10
3.2.3 Z-score limits	10
3.3 The volatility engine (Layer 0 input)	10
3.3.1 Realized Kernel (Barndorff-Nielsen et al. 2008)	11
3.3.2 Adaptive bandwidth (H^*)	11
3.3.3 Time-varying RK weights	11
3.3.4 EGARCH(1,1) with Student-t innovations	11
3.3.5 MZ R^2 -weighted blending — properly attributed	12
3.3.6 Adaptive jump detection	12
3.3.7 DVOL integration	12
3.3.8 Cross-asset beta	13
3.3.9 EGARCH/RV divergence clamp	13
3.4 Layers 1–4 — Calibration cascade	13
3.4.1 Layer 1 — CalibrationEngine (Platt $\rightarrow$ Beta $\rightarrow$ BLR auto-promotion)	13
3.4.2 Layer 2 — cal_mlp v1.1 (per-asset MLP residual + Mondrian conformal)	14
3.4.3 Layer 3 — Per-asset market blend	15
3.4.4 Layer 4 — P4.1 band-calibrated sizing	16
3.5 Layer 4 outputs: the final probability used for sizing	16
3.6 Sanity checks at the boundary	17
Part IV: Edge Detection, Sizing, and Execution	18
4.1 Fee model	18
4.2 Fee-adjusted edge	18
4.3 Position sizing	19
4.3.1 Edge tiers	19
4.3.2 Drawdown-scaled sizing (Grossman-Zhou)	20
4.3.3 Per-asset and per-product overrides	20
4.3.4 STC sizing scaler	21
4.4 Execution	21
4.4.1 Maker-first	21
4.4.2 Three-tier post_only rejection handler	21
4.4.3 STC-conditional execution	21
4.4.4 Per-asset execution overrides	22
4.4.5 Amend over cancel-replace	22

4.4.6 Fill detection	22
4.4.7 Crash recovery	22
4.5 Conditional overlays	22
4.5.1 Decided contracts (DC)	22
4.5.2 Terminal momentum (TM)	23
4.5.3 Low-price near-expiry (LPNE)	23
4.5.4 Weekend discount	23
4.5.5 Overnight discount	23
4.5.6 Loss-burst cooldown	23
Part V: Multi-vertical Engines	24
5.1 S&P 500 intraday (KXINXU series)	24
5.1.1 Equity-adapted EGARCH	24
5.1.2 Intraday seasonal deseasonalization	24
5.1.3 Configuration	24
5.1.4 HAR-RV shadow	25
5.2 Weather temperature (currently observation-only after kill)	25
5.2.1 Cities	25
5.2.2 Ensemble probability model	25
5.2.3 Bias correction	26
5.2.4 Kill rationale (2026-05-16)	26
5.3 Sports comebacks (observation, basketball alpha detected)	26
5.3.1 Leagues	26
5.3.2 Bayesian comeback model	26
5.3.3 Per-sport CalEngines	27
5.3.4 Status	27
5.4 Hourly crypto (disabled)	27
5.4.1 Kill rationale	27
5.4.2 Pre-kill data	27
5.4.3 Re-enable path	28
Part VI: The Data Corpus	28
6.1 The thesis	28
6.2 Bronze format	28
6.3 Partition scheme	29
6.4 Rotation cadence	30
6.5 Upload contract — atomic-rename + rclone copy + verify + delete	30
6.6 S3 lifecycle policy	31
6.7 Collector isolation contract (load-bearing)	31
6.8 kalshi_wire/ — the shared transport layer	33
6.9 Day-zero and current state	34
6.10 Silver and gold — planned	34
Part VII: The Agentic-Engineering Operating Model	34
7.1 Motivation	34
7.2 The verification gap	35
7.3 The Pillars	35
7.3.1 Pillar 2 — Paved roads (hooks + skills)	35
7.3.2 Pillar 3 — Equivalence snapshots	36
7.3.3 Pillar 4 — TDD-first	36
7.3.4 Pillar 5 — Tiered test suite + testmon + mutmut	37

7.3.5 Adversarial review (the distinctive pattern)	38
7.4 Import-linter contracts as architecture fitness functions	38
7.5 Shadow → observation → live promotion pipeline	39
7.6 The auto-research layer	40
7.6.1 Future: closing the auto-research loop	40
7.7 What the operating model enables	40
7.8 What the operating model does not solve	41
Part VIII: Testing Infrastructure	41
8.1 Test tier overview	41
8.2 Tier 1 — contracts	42
8.3 Tier 2 — integration	42
8.4 Tier 3 — unit	43
8.5 Tier 4 — equivalence	43
8.6 Out-of-band — mutation testing (mutmut)	43
8.7 Test selection — testmon	44
8.8 The TDD-with-agents loop in practice	44
8.9 Regression tests as failure-mode memory	44
Part IX: Infrastructure and Operations	45
9.1 Production VPS	45
9.2 Systemd units	45
9.3 Deploy pipeline	46
9.4 SQLite and journals	46
9.5 Watchdogs and alerts	47
9.6 Crash recovery	47
9.7 Auto-research crons	47
9.8 The skill catalog (operator-facing)	48
Part X: Roadmap and Future Plans	49
10.1 Near-term (next 30 days)	49
10.2 Cal_mlp v2 and v3	49
10.3 Silver and gold	49
10.4 SPX re-promotion	50
10.5 Hourly re-enablement	50
10.6 Closing the auto-research loop	50
10.7 Selective hire trajectory	51
10.8 Open problems	51
10.8.1 Recently closed (historical context)	51
10.9 The corpus-as-substrate vision	52
Part XI: References	52
Academic — volatility, calibration, distributions	52
Academic — Kelly, drawdown, position sizing	53
Industrial — agentic engineering, software architecture	53
Industrial — data architecture	54
Operational — Kalshi documentation	54
Background reading (not cited inline in this technical paper; listed for the interested reader)	54

Part I: Introduction

1.1 Scope of this document

This is the technical whitepaper. It is the deepest of the three companion documents (the others are a layperson summary and an investor whitepaper) and is intended for readers who want every implementation detail — the volatility math, the calibration pipeline architecture, the testing infrastructure, the data-corpus internals, the agentic-engineering operating model, and the roadmap. Citations are inline footnotes; primary references are listed at the end.

The system is a quantitative trading platform that operates on **Kalshi**, the only CFTC-regulated Designated Contract Market for event contracts in the United States.¹ The primary live business is short-duration cryptocurrency contracts (15-minute windows on BTC, ETH, SOL, XRP, HYPE, DOGE, BNB — all seven live post P2.4 promotion 2026-05-19), augmented by conditional overlays (decided contracts, terminal momentum, low-price near-expiry, weekend and overnight discount entries, loss-burst cooldown). Adjacent verticals — S&P 500 intraday, weather temperature across 19 U.S. cities, sports comebacks across 28 leagues — run in observation mode. Hourly crypto is currently disabled, with the re-enable path preserved.

Two additional first-class assets coexist with the trading system: a **Data Corpus** (byte-exact Web-Socket frame capture into S3, live since 2026-05-17, designed to be a permanent immutable archive) and an **agentic-engineering operating model** (one founder + Claude agents, structured around named Pillars 2–5 and adversarial-review-to-convergence). Both are covered in their own parts of this document.

1.2 Operational state at time of writing

Production state, 2026-05-19:

- `OBSERVATION_MODE` = False — live trading with real capital, continuous since 2026-02-22.
- `kalshi-bot.service` active on DigitalOcean (45.55.181.30), Ubuntu 24.04, 2 vCPU, 2 GB RAM, no swap.
- `kalshi-collector.service` active since 2026-05-17 09:57:59 UTC after an operator-initiated restart; bronze day-zero (first non-empty chunk in S3) was ~09:52 UTC during an earlier run cycle on the same day.
- Seven 15M live assets: BTC (88¢+), ETH (90¢+ main tier with a 75–79¢ sub-tier capped at 50 contracts), SOL (86¢+, taker-first), XRP (92¢+), HYPE (90¢+), DOGE (85¢+), BNB (90¢+).
- P4.1 band-calibrated sizing live; soak through 2026-05-31.
- ~6,506 tests across ~270 test files organized in 4 deploy-blocking in-tree tiers (plus the non-blocking research tier and out-of-band mutation testing).

¹Kalshi was granted DCM status by the CFTC in November 2020 and publicly launched in 2021. Per the CEA, DCM operators must satisfy 23 Core Principles covering surveillance, financial integrity, position-reporting, anti-manipulation, and rulebook compliance. Sources: [Kalshi Market Integrity](#); [Britannica](#); [CRS IF13187](#).

1.3 First-principles statement of the system

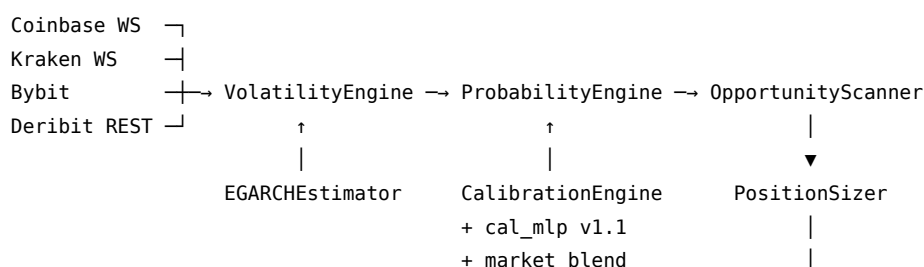
The system is built around five first-principles commitments. Each subsequent design decision in this document derives from one of these.

1. **Markets are heterogeneous information aggregators that occasionally misprice.** The bot's job is not to predict price; it is to identify when the market's *implied probability* diverges from a *better-informed probability estimate* by a fee-exceeding margin, and to take small, sized bets when it does.
2. **Edge has a half-life.** Any specific technique that produces edge will be replicated by competitors. The system is designed to (a) extract current edges efficiently, (b) ship new edges fast through a disciplined shadow-promote pipeline, and (c) preserve every microsecond of source data so future edge research has the substrate it needs.
3. **Calibration matters more than accuracy for sizing.** The literature is explicit on this point.² A model that is consistently 70% accurate but well-calibrated outperforms a model that is 75% accurate but poorly-calibrated when both are sized using Kelly. The calibration pipeline (§3) is intentionally over-engineered.
4. **Verification is the binding constraint, not generation.** This applies to both trading (the bot must verify its model probabilities post-settlement) and to engineering (the human operator must verify what AI agents produce). The architecture pushes verification into automation wherever possible: equivalence snapshots, contracts in CI, adversarial review, regression tests on every observed bug.
5. **Bronze captures everything, filter at read.** No raw data is dropped at ingest. Every Web-Socket frame is preserved byte-exact for future re-interpretation. Filtering, normalization, and feature engineering are downstream operations against immutable raw data.

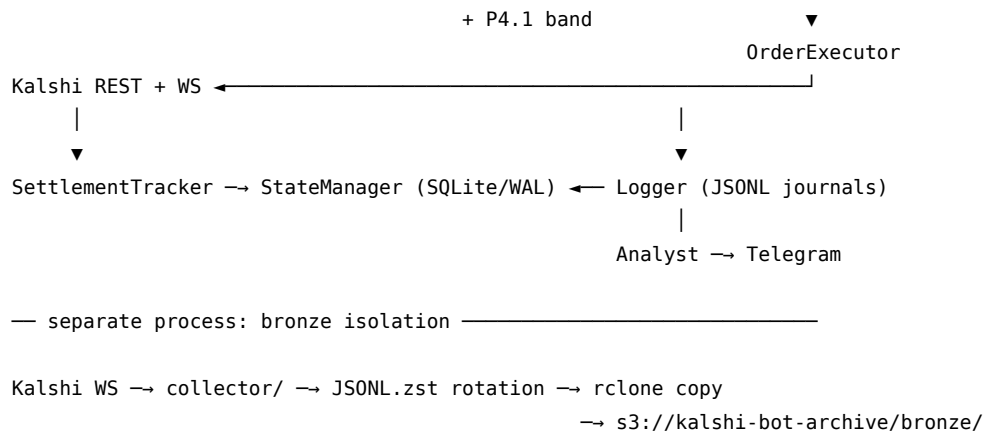
These five commitments are surprisingly load-bearing. Most of the unusual architectural choices in the rest of this document trace back to one of them.

Part II: System Architecture

2.1 Overview



²Wunderlich, F. & Memmert, D. (2023). "Machine learning for sports betting: should forecasting models be optimised for accuracy or calibration?" [arXiv:2303.06021](https://arxiv.org/abs/2303.06021). Calibration-optimized models yielded +34.7% ROI vs. -35.2% for accuracy-optimized in head-to-head testing.



The trading bot and the data collector run as separate systemd units, in separate Python processes, with separate API keys, separate disk paths, and zero shared imports. Both consume the Kalshi WebSocket via a shared *transport-only* library `kalshi_wire/` (RSA-PSS authentication, connect/reconnect logic, frame ingress; frame parsing is opt-in via the `parse_on_demand` kwarg — bot keeps the default `False` and gets parsed frames, collector opts in to `True` post P1-B-brutalist Phase B1 and uses substring scans on the raw bytes instead). The trading bot’s `bot/feeds/kalshi.py` and the collector’s `collector/ws_connection.py` consume the wire independently. This is the “two sides of the same coin” architectural amendment of 2026-05-16 that prevents drift in how Kalshi frames are received at the wire across the two pipelines (byte-identical `Frame.raw` to both — pinned by a differential test).

2.2 Source tree

The project tree has three top-level Python packages (`bot/`, `collector/`, `kalshi_wire/`), plus operational and research directories. Two of the three packages have an `__main__.py` entrypoint (`python -m bot`, `python -m collector`); `kalshi_wire/` is a transport-only library with no runnable entrypoint.

```

kalshi-bot/
├── bot/                                ← trading bot (production)
│   ├── __main__.py                    ← entrypoint shim (sacred boundary – no logic)
│   ├── _thread_env.py                 ← OMP_NUM_THREADS=1 must load FIRST
│   ├── main_loop.py                   ← MainLoop – 1Hz observation loop
│   ├── boot.py                        ← startup sequencing
│   ├── constants.py                   ← all configuration constants (canonical home)
│   ├── runtime_config.py              ← config dual-probe (constants → bot.config)
│   ├── kalshi_client.py               ← REST API client (RSA-PSS auth, rate limiting)
│   ├── state.py                       ← StateManager – SQLite-backed positions/orders/fills
│   ├── scanner/                       ← OpportunityScanner – multi-stage filter pipeline
│   ├── executor.py                    ← OrderExecutor – three-tier rejection handling
│   ├── settlement.py                  ← SettlementTracker – 30s polling
│   ├── order_flow.py                  ← KalshiOrderFlowTracker – shadow signals
│   ├── orphan_db_watchdog.py          ← startup orphan-PID detection
│   ├── engines/                       ← per-domain probability engines
│   │   ├── volatility.py              ← VolatilityEngine (RK + EGARCH + MZ blend + jumps)
│   │   └── probability.py             ← ProbabilityEngine (NIG CDF + calibration + blend)

```

```

| | | calibration.py      ← CalibrationEngine (Fixed → Platt → Beta → BLR)
| | | spx_engine.py       ← SPX equity-adapted EGARCH
| | | weather_engine.py   ← ensemble Gaussian fit + bias correction
| | | sports_engine.py    ← Bayesian comeback model
| | |   sports_data.py    ← per-sport LR tables + state-shape helpers
| | | feeds/              ← market data feeds
| | |   coinbase.py       ← spot price WS (7 assets – BNB added T1 2026-05-17, 5-min buffer + 15-
hour EGARCH)
| | |   cross_exchange.py ← Binance/Kraken/Bybit WS (CrossExchangeFeed)
| | |   kalshi.py         ← Kalshi WS feed (consumes kalshi_wire)
| | |   orderbook_schema.py
| | | fetchers/           ← REST pollers extracted as their own layer (deribit.py, coinglass.py). Engine-
internal REST polling for Polygon/Finnhub/ESPN/Open-Meteo lives inside each engine module.
| | |   helpers/          ← shared utilities
| | |   | derived_features.py ← canonical feature transforms (lock-step with scripts/cal_mlp/)
| | |   |   band_calibration.py ← P4.1 42-cell hierarchical-shrunk lookup
| | |   | snapshots/        ← dashboard state + Supabase sync
| | |   | ai/               ← Claude API integration
| | |   |   analyst.py      ← Analyst – per-loss RCA
| | |   |   auditor.py      ← scheduled audit-summary LLM dispatcher
| | |   |   researcher.py   ← auto-research scheduled jobs(3x daily)
| | |   | notifier.py       ← TelegramNotifier
| | |   |   logger.py       ← JSONL journals (append-only, daily rotation)
| | |
| | | collector/          ← data corpus collector (separate process)
| | |   | __main__.py
| | |   | main_loop.py      ← collector run loop, multi-conn fan-out
| | |   | ws_connection.py  ← BronzeArchiver – WS frame ingestion to writer
| | |   | rest_snapshot.py  ← Kalshi REST hourly catalog refresh
| | |   | subscription_manager.py ← per-tier conn assignment
| | |   | writer.py         ← in-flight JSONL writer + rotation
| | |   | uploader.py       ← rclone copy + verify + delete-local
| | |   |
| | |   | (collector consumes kalshi_wire/auth for RSA-PSS – no separate auth.py here)
| | |
| | | kalshi_wire/        ← shared transport (pure-leaf package)
| | |   | auth.py          ← RSA-PSS-SHA256 signature construction + REST header builder
| | |   | ws_client.py      ← WSClient – connect/reconnect/silence-watchdog + frame parse (optional via `parse_on_deman
P1-B-brutalist Phase B1 to skip parse on the asyncio thread) + 6-field envelope construction
| | |
| | | ops/                ← deployment artifacts
| | |   | kalshi-bot.service ← systemd unit (live)
| | |   | kalshi-collector.service ← systemd unit (live)
| | |   | install.sh        ← multi-unit installer
| | |   | watchdog.py       ← cross-instance orphan watchdog
| | |   | ...
| | |
| | | scripts/            ← off-VPS tooling
| | |   | cal_mlp/          ← cal_mlp v1.1 training + serving recipe
| | |   | backfill/         ← historical data backfills
| | |   | audit/            ← /audit skill scripts (per-vertical Wilson CI)
| | |   | ops/              ← maintenance crons (collector_health_monitor.py, etc.)
| | |
| | | tests/              ← ~6,506 tests, ~270 files, multi-tier
| | |   | contracts/        ← Tier 1: API + signature contracts

```

```

|   ├── integration/           ← Tier 2: cross-module flow
|   ├── unit/                 ← Tier 3: function-level
|   ├── equivalence/          ← Tier 4: pinned engine outputs (test_volatility_engine/ + test_probability_engine/)
|   ├── regression/           ← failure-mode-memory regression tests
|   ├── hooks/                ← Claude Code hook integration tests
|   └── fixtures/              ← shared test fixtures
|   (mutmut runs out-of-band against bot/engines/{volatility,probability}.py with the equivalence tests as runner –
no separate tier directory)
|
|   ├── kb/                    ← local-only knowledge base
|   |   ├── _index.md
|   |   ├── decisions/         ← architectural decision records
|   |   ├── findings/          ← research outputs
|   |   ├── failures/          ← postmortems (L1 – L100+ named lessons)
|   |   ├── concepts/          ← reusable design vocabulary
|   |   └── strategies/         ← strategy specifications
|   |
|   ├── agent_docs/            ← canonical living-truth documents
|   |   ├── current_state.md
|   |   ├── config_reference.md
|   |   ├── db_schema.md
|   |   ├── bot_layout.md
|   |   ├── calibration_pipeline.md
|   |   └── p4_1_calibration_baseline.md
|   |
|   ├── .claude/               ← Claude Code skills, agents, hooks
|   |   ├── skills/             ← user-invocable workflows (/deploy, /audit, etc.)
|   |   ├── agents/             ← specialized agents (adv-reviewer, drift-sweeper, ...)
|   |   └── templates/          ← scaffolds for new skills/agents/audits
|   |
|   ├── docs/whitepaper/       ← this document + investor + layperson
|   ├── .importlinter           ← architecture fitness functions
|   ├── CLAUDE.md               ← root agent context (auto-loads)
|   ├── AGENTS.md               ← symlink to CLAUDE.md (two-file-mode)
|   └── start.sh                 ← bot launcher

```

The discipline `bot/__main__.py` is a sacred-boundary shim with no logic is enforced by the root `CLAUDE.md` rule + the Bit 9.3-iii.c deletion of `bot/_impl.py` (pinned by `tests/integration/test_bit_9_3_iii_c_impl_deletion.py`). Logic lives in the submodules — never in the entrypoint.

2.3 Runtime topology

```

systemd ───┬── kalshi-bot.service      (ExecStart=start.sh → python -m bot)
           └── kalshi-collector.service (ExecStart=collector-start.sh → python -m collector)

```

The bot's `start.sh` activates the venv and runs `python -m bot`. The collector's `collector-start.sh` sources a dedicated `.env.collector` file (separate from the bot's `.env`) and runs `python -m collector`.

`OMP_NUM_THREADS=1` is critical and is set by `bot/_thread_env.py` which **must** import before any numpy-dependent module. Without it, numpy's transitive parallelism would oversubscribe the 2

vCPU budget (the kernel scheduler floats both bot and collector across the two vCPUs; the collector's Nice=10 polite-background priority is the load-bearing isolation knob keeping the bot's scan ticks responsive) and degrade real-time performance.

2.4 Data flow

The bot's MainLoop runs at 1 Hz. Each tick:

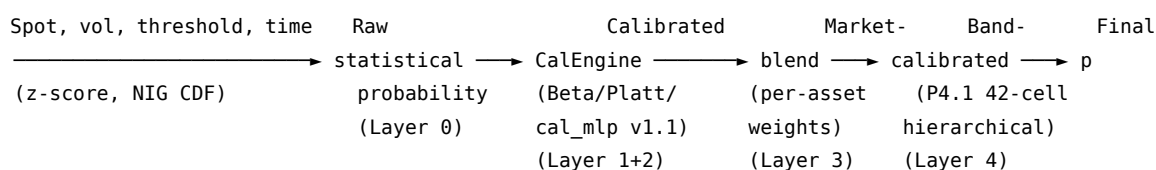
1. Read latest spot from CoinbaseFeed (5-min buffer for current state; 15-hour return buffer for EGARCH fits).
2. Update VolatilityEngine with new returns; recompute blended volatility.
3. Discover active 15M windows via `discover_active_windows()` cross-referenced with `product_type`.
4. For each active market: scanner pipeline (probability estimation → calibration → fee-adjusted edge → multi-stage filter → sizing → execution decision).
5. Order updates from Kalshi WebSocket are processed asynchronously in a separate thread; fills update StateManager and notify MainLoop on the next tick.
6. SettlementTracker polls every 30 seconds; settled outcomes route to the appropriate per-product CalEngine and update calibration state.
7. Analyst examines every losing trade post-settlement and emits a Claude-API-generated root-cause analysis when high-confidence.

The collector's MainLoop runs asynchronously (no fixed cadence). Each WS frame received from `kalshi_wire.WSClient` is dispatched to a `BronzeArchiver` instance for that connection-channel. The archiver writes to an in-flight `.jsonl` file; rotation triggers on either a 5-minute timer or a 100 MB size threshold; rotated chunks are `zstd`-compressed, uploaded via `rcclone`, verified, and deleted locally.

Part III: Probability and Calibration Pipeline

3.1 The four-stage probability cascade

The system produces a final, sizing-ready probability estimate through a four-stage cascade:



Each layer is independently testable, independently regressed, and independently overrideable. Layers can be bypassed via configuration if a regression appears at any stage.

3.2 Layer 0 — Raw statistical probability

3.2.1 Z-score construction

Given current spot S , contract threshold T , blended volatility σ_{blended} (5-second scale), and seconds remaining t , the z-score is:

$$z = \frac{T - S}{S \cdot \sigma_{\text{blended}} \cdot \sqrt{t/5}}$$

The denominator represents the expected magnitude of price movement over the remaining window. As $t \rightarrow 0$, this denominator shrinks toward zero and $|z|$ can blow up — which is precisely why the `spot_distance_to_strike_sigma` feature (a `cal_mlp` input) is winsorized at ± 25 (see §3.4.2).

3.2.2 NIG CDF

Raw probability is the upper-tail integral of the Normal Inverse Gaussian distribution³:

$$p_{\text{raw}} = 1 - F_{\text{NIG}}(z; \alpha, \beta, \mu, \delta)$$

NIG parameters per asset are fit by MLE on 7 days of 60-second returns (~10,080 samples per asset) and stored in `dist_config.json`. Refit cadence: daily. **Why NIG over Student-t:**

- **Asymmetry:** β captures the empirical skew in crypto returns. Example: BTC $\beta \approx -0.019$ (slight left skew). Student-t is symmetric; it cannot represent asymmetric tails.
- **Tail fit:** KS-test p-values are substantially higher for NIG (BTC ≈ 0.11 , ETH ≈ 0.42) vs. Student-t (effectively 0). NIG accommodates the empirical return distribution; Student-t is rejected at any conventional significance level.

If NIG parameters are unavailable (file missing, asset added before refit), the engine falls back to Student-t(df=4) with a logged warning.

3.2.3 Z-score limits

If $|z| > 25$, the candidate is rejected as data-corruption-suspect. Previously the limit was 12, which was blocking valid high-conviction trades; postmortem analysis of historical losses showed every losing trade had $|z| < 25$, so the limit was raised to that empirical bound.

3.3 The volatility engine (Layer 0 input)

The volatility engine produces σ_{blended} — a 5-second-scale realized volatility estimate that feeds Layer 0. It combines four academic techniques with adaptive blending.

³Barndorff-Nielsen, O. E. (1997). “Normal Inverse Gaussian Distributions and Stochastic Volatility Modelling.” *Scandinavian Journal of Statistics* 24(1): 1–13. The standard parameterization is $(\alpha, \beta, \mu, \delta)$: α controls tail heaviness, β controls asymmetry/skew, μ is location, δ is scale. The PDF is a variance-mean mixture of a Normal with an inverse-Gaussian mixing distribution.

3.3.1 Realized Kernel (Barndorff-Nielsen et al. 2008)

Standard sample variance of high-frequency returns is biased by market microstructure noise (bid-ask bounce, discrete tick sizes). The Realized Kernel estimator⁴ corrects this using a kernel-weighted autocovariance function:

$$RK = \sum_{h=-H}^H k\left(\frac{h}{H+1}\right) \gamma_h, \quad \gamma_h = \sum_{j=|h|+1}^n r_j r_{j-|h|}$$

where $k(\cdot)$ is the Parzen kernel (smooth at the origin — $k'(0) = k'(1) = 0$ — which guarantees non-negative RK). H is the bandwidth.

3.3.2 Adaptive bandwidth (H^*)

Rather than fixing $H=1$, the engine estimates the optimal bandwidth from the data:

$$H^* = c \cdot \left(\frac{\hat{\omega}^2}{IV} \right)^{2/5} \cdot n^{3/5}$$

where ω^2 is the estimated microstructure noise variance, IV is integrated variance, and the Parzen-kernel constant $c \approx 3.5134$. This produces tighter estimates during calm periods (low noise-to-signal) and wider smoothing during noisy periods.

3.3.3 Time-varying RK weights

Multiple RK estimators at different scales (5s, 15s, 1min) are blended using time-varying weights that adapt to current market conditions rather than fixed proportions. The MZ-R²-weighted blending below applies here as well.

3.3.4 EGARCH(1,1) with Student-t innovations

The EGARCH model⁵ captures volatility clustering and the asymmetric news-impact effect:

$$\log(\sigma_t^2) = \omega + \alpha(|z_{t-1}| - E[|z|]) + \gamma z_{t-1} + \beta \log(\sigma_{t-1}^2)$$

Fitted by maximum likelihood on 10,800 samples (15 hours at 5-second intervals) with Student-t innovations (df typically 3.2–3.8 for crypto). Refit cadence: every 2 hours.

⁴Barndorff-Nielsen, O. E., Hansen, P. R., Lunde, A., & Shephard, N. (2008). “Designing Realized Kernels to Measure the ex post Variation of Equity Prices in the Presence of Noise.” *Econometrica* 76(6): 1481–1536. DOI 10.3982/ECTA6495. The implementation companion is BNHLS 2009, *Econometrics Journal*, “Realized Kernels in Practice: Trades and Quotes.”

⁵Nelson, D. B. (1991). “Conditional Heteroskedasticity in Asset Returns: A New Approach.” *Econometrica* 59(2): 347–370. Nelson’s original used GED innovations; the Student-t variant is a common practitioner extension. Note: technically EGARCH delivers *asymmetric news impact*, not strict leverage (which requires return-volatility innovation correlation per the financial definition). The common conflation is fine for practitioner usage but academically imprecise.

The negative γ parameter captures the empirical pattern that negative shocks raise volatility more than positive shocks of equal magnitude. For crypto, bounds are $\gamma \in (-0.15, -0.02)$; for SPX equities, bounds are $\gamma \in (-0.30, -0.05)$, reflecting the stronger leverage effect in equity returns.

3.3.5 MZ R²-weighted blending — properly attributed

The engine blends EGARCH and RK forecasts using weights derived from each estimator's Mincer-Zarnowitz regression R²:

$$RV_{t+1} = \alpha + \beta \hat{\sigma}_t + \varepsilon_t$$

The R² from this regression measures forecast quality.⁶ Weights smoothed via EMA ($\lambda=0.97$). **Important attribution:** the Mincer-Zarnowitz regression is an *evaluation* device. The use of R² as a *combination weight* is an industrial choice, related but not identical to the canonical Bates-Granger (1969)⁷ inverse-MSE forecast combination. We do not claim Mincer-Zarnowitz invented combination weighting; we use their evaluation regression to drive a Bates-Granger-style combination.

Below an R² threshold of 0.10, the system reverts to equal-weight blending.

3.3.6 Adaptive jump detection

Returns above an adaptive threshold are flagged as jumps and excluded from RK estimation for a cooldown window. The threshold is **percentile-based** rather than fixed- σ — the asset's own running volatility distribution at the 99.9th percentile defines the trigger. This is operationally distinct from named academic jump tests (Lee-Mykland 2008⁸, BNS bipower 2006⁹); we describe it as a percentile-based practitioner heuristic, not a formal jump test.

3.3.7 DVOL integration

When Deribit's DVOL implied volatility (BTC, ETH only — SOL/XRP/HYPE/DOGE/BNB have no public IV index) diverges materially from realized, the engine blends in the implied estimate using inverse-variance weighting. This respects forward-looking information during regime changes while anchoring to observed data.

⁶Mincer, J. A. & Zarnowitz, V. (1969). "The Evaluation of Economic Forecasts." In J. Mincer (ed.), *Economic Forecasts and Expectations*. NBER / Columbia University Press, pp. 3–46.

⁷Bates, J. M. & Granger, C. W. J. (1969). "The Combination of Forecasts." *Operational Research Quarterly* 20(4): 451–468. The canonical academic survey is Timmermann (2006), "Forecast Combinations," *Handbook of Economic Forecasting* Vol. 1, Ch. 4.

⁸Lee, S. S. & Mykland, P. A. (2008). "Jumps in Financial Markets: A New Nonparametric Test and Jump Dynamics." *Review of Financial Studies* 21(6): 2535–2563. Threshold-based using a rolling local-volatility estimator and Gumbel-tail critical values.

⁹Barndorff-Nielsen, O. E. & Shephard, N. (2006). "Econometrics of Testing for Jumps in Financial Economics Using Bipower Variation." *Journal of Financial Econometrics* 4(1): 1–30. Compares realized variance against bipower variation; the ratio z-statistic is the BNS jump test.

3.3.8 Cross-asset beta

For assets without direct DVOL data (SOL, XRP, HYPE, DOGE, BNB), the system estimates a cross-asset beta against BTC using a 60-return lookback window, clamped to [0.5, 3.0]. This allows derivative-implied signals to propagate across correlated assets via:

$$\sigma_{\text{asset}}^{\text{implied}} \approx \beta_{\text{asset,BTC}} \cdot \sigma_{\text{BTC}}^{\text{implied}}$$

3.3.9 EGARCH/RV divergence clamp

If the ratio of EGARCH-forecast variance to RK-realized variance falls outside $[1/3, 3]$, EGARCH is rejected and the engine falls back to RK-only volatility for that tick. This is a structural safety guard against EGARCH numerical instability during regime breaks.

3.4 Layers 1–4 — Calibration cascade

The raw probability p_{raw} from Layer 0 is corrected by per-product calibration engines that learn from settlement outcomes.

3.4.1 Layer 1 — CalibrationEngine (Platt → Beta → BLR auto-promotion)

Each market type (15M crypto, weather-per-city, sports-per-group, SPX-D, hourly) maintains its own CalibrationEngine instance in a registry (`_CAL_REGISTRY[product_key]`). The engine maintains an internal method-selection state:

Method	Min samples	Description
Fixed logistic ($\beta=0.85$)	0	Default fallback — compresses extreme probabilities
Platt scaling	200	2-parameter logistic (A, B) fitted to outcomes ¹⁰
Beta calibration	350	3-parameter Kull/Silva Filho/Flach 2017 ¹¹
Bayesian Linear Regression	50	Bayesian approach with credible intervals

¹⁰Platt, J. C. (1999). “Probabilistic Outputs for Support Vector Machines and Comparisons to Regularized Likelihood Methods.” *Advances in Large Margin Classifiers* (MIT Press), pp. 61–74. The Lin/Lin/Weng (2007) numerically-stable implementation is the standard reference for production use.

¹¹Kull, M., Silva Filho, T. & Flach, P. (2017). “Beta calibration: a well-founded and easily implemented improvement on logistic calibration for binary classifiers.” *AISTATS 2017*, PMLR 54:623–631. Extended version in *Electronic Journal of Statistics* 11(2): 5052–5080.

Auto-promotion: the engine recomputes its Brier score on a holdout set at each method-promotion threshold and adopts the better method. The pattern of “promote calibration method as sample size accumulates” is a practitioner pattern (no single canonical paper), grounded in Niculescu-Mizil & Caruana (2005) finding that isotonic and Beta-style methods require ~1000+ samples to outperform Platt.¹²

Current 15M production state: passthrough mode. The 15M CalibrationEngine has trained Beta and BLR fits available but the engine selects passthrough because raw probability’s measured Brier on the held-out sample is *lower* than the BLR-fit probability’s. We do not force a less-accurate method just because it has more parameters. Per-city weather, per-sport-group, and SPX-D engines run their full pipelines.

3.4.2 Layer 2 — cal_mlp v1.1 (per-asset MLP residual + Mondrian conformal)

cal_mlp v1.1 is a per-asset multilayer perceptron residual calibrator wrapped in a Mondrian conformal predictor¹³ for distribution-free group-conditional coverage. Architecture:

- **Inputs:** 8 continuous features (market_price, prob_breakeven_gap, seconds_to_close, time_decayed_proximity, hour_sin, hour_cos, spot_distance_to_strike_sigma, abs_spot_distance_to_strike_sigma)+categorical embeddings(price_tier, stc_bucket, vol_regime_int, side_int).
- **Architecture:** small MLP (3 hidden layers, sizes 64-32-16), ReLU activations, sigmoid output. Per-asset weight files for BTC/ETH/SOL/XRP under the v1.1_production recipe (cfg_fp 345978797274721f).
- **Serving scope:** BTC/ETH/SOL/XRP only. HYPE, DOGE, and BNB bypass cal_mlp entirely — HYPE/DOGE were promoted to live trading via a Brier-sweep raw_prob direct-promote on 2026-05-14 (cal_mlp training arc retired for the HYPE/DOGE cohort); BNB was promoted via the same path on 2026-05-19 (P2.4, 86b9zmj37). A four-feature replay_v1 recipe exists in the training tooling (scripts/cal_mlp/) for historical-replay backfill scoring during the pre-promotion T1 shadow phase, but is not wired into live serving for any asset.
- **Mondrian conformal wrapper:** prediction sets at each of (price_tier × stc_bucket × vol_regime × side) partition are computed for finite-sample coverage. This is a stronger guarantee than marginal coverage¹⁴.
- **Sigma winsorization:** spot_distance_to_strike_sigma blows up to ±3,000+ as the time denominator approaches zero (terminal STC). Without clipping, z-scoring across the column inflates standard deviation 100× and collapses real signal. Fix: features.SIGMA_WINSOR_ABS_CAP = 25.0, applied via features.apply_sigma_winsor(sd). Lock-step test pin at tests/contracts/test_calmlp_lockstep.py.
- **Bundle versioning:** each trained bundle is fingerprinted by an 8-byte hash of its canonical feature set (cfg_fp). Current production: 345978797274721f.

¹²Niculescu-Mizil, A. & Caruana, R. (2005). “Predicting Good Probabilities with Supervised Learning.” *ICML 2005*.

¹³Vovk, V. (2012). “Conditional validity of inductive conformal predictors.” *PMLR 25*. Vovk, V., Gammerman, A. & Shafer, G. (2005). *Algorithmic Learning in a Random World*. Springer.

¹⁴Vovk, V. (2012). “Conditional validity of inductive conformal predictors.” *PMLR 25*. Vovk, V., Gammerman, A. & Shafer, G. (2005). *Algorithmic Learning in a Random World*. Springer.

Lock-step contract. The `cal_mlp` pipeline has FOUR drift sites that must stay in lock-step (any change to feature transforms ships in ONE commit across all sites):

1. `scripts/cal_mlp/extract_data.py::build_feature_frame` (train)
2. `scripts/cal_mlp/post_hoc_processor.py::_process_row` (serve post-hoc)
3. `scripts/cal_mlp/integration.py::should_block_tm96` (serve sync gate)
4. `scripts/cal_mlp/sim_pnl.py` (train-time sim-PnL evaluation)

Canonical formula homes: - `bot/helpers/derived_features.py::compute_derived_features` for `spot_distance_to_strike_sigma + probabreakeven_gap` - `bot/helpers/derived_features.py::compute_ho` for `hour-of-day cyclic encoding` - `scripts/cal_mlp/features.py::compute_cfg_fp` + `SIGMA_WINSOR_ABS_CAP` + `apply_sigma_winsor`

AST + runtime parity guards at `tests/contracts/test_calmlp_lockstep.py` pin all four sites. Splitting any of these across commits would create train/serve skew — model trained on one distribution, served from another. This is the most-tested area of the codebase by line-count, reflecting the cost of a train/serve drift bug (silent re-training on contaminated distribution, surfacing as gradual edge erosion).

3.4.3 Layer 3 — Per-asset market blend

The calibrated probability is blended with the market-implied probability `best_ask/100`:

$$p_{\text{blended}} = (1 - w_{\text{asset}}) \cdot p_{\text{cal}} + w_{\text{asset}} \cdot p_{\text{market}}$$

Current production weights (canonical lockstep `MARKET_BLEND_W_BY_ASSET`):

Asset	w (market weight)	model/market split
BTC	0.10	90/10
ETH	0.20	80/20
SOL	0.80	20/80
XRP	0.90	10/90
HYPE	0.80	20/80
DOGE	0.60	40/60
BNB	0.20	80/20

These replaced a legacy 60/40 default in three atomic ships: P2.1.d (BTC/ETH/SOL/XRP, 2026-05-13), P2.3 (HYPE/DOGE, 2026-05-14), and P2.4 (BNB, 2026-05-19). Justification: a 4×6 sim-PnL sweep against `cal_mlp` v1.1 calibration showed different optimal weights per asset, driven by per-asset Brier improvements (BTC −13%, ETH −11%, SOL −3%, XRP −6%). HYPE/DOGE and BNB derived from B.1-equivalent Brier sweeps on T1 shadow data (n=1469/1710/721 respectively); BNB's argmin at `w=0.20` matches ETH's pattern — its raw model is well-calibrated, beating market by ~10%

Brier. High-Brier-improvement assets reward heavy model weighting; near-parity assets default to the market as the more reliable signal.

Doc-drift contract. This constant appears in `bot/constants.py` and is referenced across eight documentation surfaces tracked by `scripts/audit/doc_drift_check.py`: `README.md`, `whitepaper.md`, `whitepaper_investor.md`, `CLAUDE.md`, `AGENTS.md`, `agent_docs/config_reference.md`, `agent_docs/calibration_pipeline.md`, and `kb/concepts/edge-thresholds.md`. Any change must ship lockstep across all eight; make `doc-drift` catches drift in CI.

3.4.4 Layer 4 — P4.1 band-calibrated sizing

Promoted 2026-05-17. Kelly sizing receives a **band-stratified calibrated probability** rather than the blended probability directly. The implementation lives in `bot/helpers/band_calibration.py` and exposes `calibrated_prob_for_sizing(asset, best_ask, final_prob, product_type)`.

The lookup table is 42 cells: 6 assets × 7 price bands (70–79 / 80–85 / 86–89 / 90–93 / 94–96 / 97–98 / 99). Each cell stores the empirical win rate of trades that fell in that cell, computed from settled outcomes over a recent lookback window.

Hierarchical shrinkage: thin cells shrink toward the per-band aggregate prior with shrinkage parameter $k=30$. For a cell with n observations and observed win rate $\hat{p}$, the shrunk estimate is:

$$\hat{p}_{\text{shrunk}} = \frac{n \cdot \hat{p} + k \cdot \bar{p}_{\text{band}}}{n + k}$$

where $\bar{p}_{\text{band}}$ is the band-aggregate win rate across all 6 assets.

Hybrid lookback: 30 days for the volatile 70–93¢ regime (regime-sensitive), 60 days for the thin-cell 94–100¢ regime (stability matters more than freshness).

Precision: 6 decimal places on the empirical rate.

Scope: only 15M Kelly sizing. V2 sizing (`_v2_prob`) and NO-side (`no_prob`) are explicitly pass-through. Hourly/SPX/weather are pass-through. **Trade selection gates are unchanged** — only Kelly *magnitudes* shift.

Soak: 14d band-stratified through 2026-05-31. Rollback rule: per-(asset × band) realized rate ±5pp of baseline. Baseline frozen 2026-05-17 09:35 UTC against VPS HEAD `e3aecd4`; sidecar at `agent_docs/p4_1_calibration_baseline.md`.

Per-cell escape: `env var BAND_CALIBRATION_DISABLED_CELLS=BTC:97,HYPE:80` disables specific cells without rolling back the whole feature.

3.5 Layer 4 outputs: the final probability used for sizing

```
# Conceptual flow
p_raw = NIG_cdf_upper_tail(z_score)           # Layer 0
p_cal = cal_engine.predict(p_raw)             # Layer 1
```

```

# 15M: passthrough (raw
    ↳ beats fits on
    ↳ Brier)
# weather/sports/SPX-D:
    ↳ full Beta/BLR
    ↳ pipeline
    ↳ active

p_calmlp = cal_mlp_v1_1.predict(features, asset)

# Layer 2
# BTC/ETH/SOL/XRP:
    ↳ active (v1.1
    ↳ production)
# HYPE/DOGE: bypassed
    ↳ (cal_mlp arc
    ↳ retired for
    ↳ them;
    ↳ promoted via
    ↳ raw_prob
    ↳ direct-
    ↳ promote
    ↳ 2026-05-14)
# weather/sports/SPX-D:
    ↳ not used
    ↳ (Layer 2 is
    ↳ 15M-specific)

p_blended = (1 - w[asset]) * p_calmlp_or_p_cal + w[asset] * (best_ask / 100) # Layer 3
p_sizing = calibrated_prob_for_sizing(asset, best_ask, p_blended, # Layer 4
                                     product_type)
# trade decision uses p_blended; size decision uses p_sizing

```

This separation — trade selection at Layer 3, Kelly sizing at Layer 4 — is structurally important. The P4.1 design specifically did not change trade selection (which would have been a multi-week regression effort) but did change sizing (a tractable, easily-rollback-able change with clear measurement criteria).

3.6 Sanity checks at the boundary

Three model-output sanity checks operate at the boundary between Layer 4 and the position sizer:

1. **Z-score limit** (§3.2.3): $|z| > 25$ rejects the candidate.
2. **Model-market discrepancy**: if $p_{cal} > 90\%$ but $best_ask < 75¢$, the candidate is rejected — the model is suspected of missing material information.
3. **EGARCH/RV divergence clamp** (§3.3.9): if EGARCH/RK ratio falls outside $[1/3, 3]$, RK-only volatility is used for that tick.

Each check is a separate gate with a separate test pin in CI.

Part IV: Edge Detection, Sizing, and Execution

4.1 Fee model

Per the bot's current configuration (matching `agent_docs/current_state.md`):

$$\text{taker fee} = \lceil 0.07 \cdot C \cdot P \cdot (100 - P) / 100 \rceil \text{ cents (per fill)}$$

$$\text{maker fee} = 0$$

where C is contract count and P is trade price in cents. The ceiling applies to the *total* fee on the fill, not per contract. The $P \times (100 - P)$ curvature is load-bearing — fees collapse near 0 and 100, exactly where the system trades most often (near-100¢ for high-conviction longs).

SPX (“finance” category) uses 0.035 taker (half crypto rate).

Note on maker-fee schedule. Kalshi's public fee schedule¹⁵ effective February 2026 documents a maker fee of approximately $\text{ceil}(0.0175 \times C \times P \times (100 - P) / 100)$ cents (one-fourth of the taker rate, same dimensional form). The bot currently operates on the assumption of maker = \$0, which may reflect product-category-specific rules for 15M crypto contracts or a stale assumption requiring verification. This discrepancy is a tracked followup; if maker fees apply to the bot's products, the edge calculation in §4.2 understates total trading cost by approximately 0.4¢ per maker fill at typical 90¢ prices.

The scanner evaluates edge using **taker fees worst-case**, so any candidate that passes the filter remains profitable even if the maker leg is rejected and escalation falls to a taker IOC.

4.2 Fee-adjusted edge

$$\text{edge} = p_{\text{blended}} - \frac{\text{best_ask}}{100} - \frac{\text{taker_fee}}{C \cdot 100}$$

A candidate is accepted if $\text{edge} \geq \text{get_min_edge}(\text{best_ask})$. The minimum-edge schedule is V-shaped — lowest in the 91–92¢ region where contract reliability is highest, climbing at both extremes:

Entry price	Min edge
97¢+	1.0%
95–96¢	0.75%
93–94¢	0.5%

¹⁵Kalshi Fee Schedule; Kalshi Help Center — Fees.

Entry price	Min edge
91–92¢	0.20%
89–90¢	0.25%
80–88¢	0.25%

The V-shape is empirical, not theoretical — the bottom of the V at 91–92¢ is where post-fee win-rate volatility is lowest in the historical data. Higher prices demand more edge because the asymmetric loss (paying 97¢ to win 3¢) requires more cushion.

A flat fallback of 0.25% (MIN_EDGE_PCT) applies if the price-dependent schedule is unavailable.

4.3 Position sizing

4.3.1 Edge tiers

Fee-adjusted edge	Risk fraction of bankroll
≥ 4.0%	25%
≥ 2.5%	20%
≥ 1.8%	15%
≥ 1.2%	10%
≥ 0.9%	7%
≥ 0.7%	5%
≥ 0.5%	3%
≥ 0.25%	2%

This is **operationally** a discretized fractional-Kelly schedule with parameter-uncertainty discounting. There is no single canonical citation for “edge-tiered binning of Kelly” — the form is a practitioner pattern. The grounding citations are Kelly (1956)¹⁶ for the original log-optimal derivation and Thorp (2006)¹⁷ for the fractional-Kelly practitioner argument. We bin discretely because (a) per-trade p and b are estimated quantities with parameter uncertainty, (b) discrete tiers prevent overfitting to a particular edge measurement, and (c) discrete-contract sizing in event markets makes the continuous form non-actionable.

¹⁶Kelly, J. L., Jr. (1956). “A New Interpretation of Information Rate.” *Bell System Technical Journal* 35(4): 917–926. [PDF mirror](#).

¹⁷Thorp, E. O. (2006). “The Kelly Criterion in Blackjack, Sports Betting, and the Stock Market.” *Handbook of Asset and Liability Management* Vol. 1, pp. 385–428.

4.3.2 Drawdown-scaled sizing (Grossman-Zhou)

Balance vs. 7-day rolling cash HWM	Sizing multiplier
$\geq 85\%$	1.0× (full)
75–85%	0.5× (half)
65–75%	0.25× (quarter)
$< 65\%$	0.10× (floor — never zero)

The HWM is a 7-day rolling maximum of realized cash balance, not mark-to-market (mark-to-market HWM would be more aggressive but more sensitive to in-flight position swings). The discrete steps approximate the Grossman-Zhou (1993)¹⁸ continuous result that drawdown-constrained log-optimal allocation scales with distance from the HWM. The bottom step is a 10% floor on size, not a halt — the explicit code comment is “Floor: never fully halt.” The reasoning is that a sustained deep drawdown is exactly when continued small-size measurement matters most; a complete halt would freeze the feedback signal the system needs to detect a genuine regime break vs. a recoverable streak. An operator-driven kill switch (OBSERVATION_MODE=True or `systemctl stop kalshi-bot`) remains available for true halt.

4.3.3 Per-asset and per-product overrides

Product	Max risk per trade	Kelly fraction
15M BTC	15%	full Kelly (within tier)
15M ETH	20%	full
15M SOL	15%	full
15M XRP	15%	full
15M HYPE	10%	full
15M DOGE	10%	full
Hourly (currently disabled)	15%	quarter Kelly (0.25); per-window position limit max 2
SPX	10%	eighth Kelly (0.125)
Weather YES sim	10%	quarter Kelly (0.25)
Weather NO (killed)	1 contract fixed	—

¹⁸Grossman, S. J. & Zhou, Z. (1993). “Optimal Investment Strategies for Controlling Drawdowns.” *Mathematical Finance* 3(3): 241–276. The HWM concept itself originates in hedge-fund incentive-fee contracts per Goetzmann, Ingersoll & Ross (2003), *J. Finance* 58(4): 1685–1718.

Product	Max risk per trade	Kelly fraction
LPNE	50 contracts fixed	—
Terminal momentum	25–500 contracts cap	edge-tiered

4.3.4 STC sizing scaler

Universal across 15M: $\text{contracts} = 300 / \text{STC}$ when $\text{STC} > 300\text{s}$. This reduces position size proportionally to time remaining — less time exposure = less risk. Implemented as a wrapper around the per-tier size calculation.

4.4 Execution

4.4.1 Maker-first

Default order placement is `post_only=True` with `time_in_force="day"`. The bot's current configuration treats maker fee as \$0 (\$4.1) — Kalshi's published February 2026 fee schedule documents a non-zero maker rate of ~25% of taker for at least some product categories, but the bot operates on the \$0 assumption pending operator verification of the rate applicable to 15M crypto contracts. Either way, the maker leg avoids the full 0.07-multiplier taker rate and the per-fill ceiling, which is the load-bearing cost advantage on every fill that posts at the resting limit.

4.4.2 Three-tier `post_only` rejection handler

If maker is rejected (price locked the spread):

1. **Degraded maker** — re-attempt 1¢ worse, still `post_only=True`. Recomputes edge at the worse price.
2. **Taker IOC** — `time_in_force="immediate_or_cancel"`, taker fee. Recomputes edge at the worse price.
3. **Direct fail** — log and skip if edge becomes insufficient at the worse price.

4.4.3 STC-conditional execution

STC	Execution mode
> 180s	Maker-first with three-tier rejection handling
≤ 180s	Direct taker IOC (skip maker entirely)

Justification: empirical maker fill rates collapse at low STC. Two measurements pin the curve: at 0–60s STC, fill rate was ~7.7% (1 of 13 candidates filled — the legacy measurement when the threshold

was 60s). The subsequent 75→180s raise measured 0% fill (26 of 26 maker orders escalated to taker). The 180s threshold combines both regimes — at any STC below it, direct taker is strictly better than waiting for an unlikely maker fill.

4.4.4 Per-asset execution overrides

- **SOL**: taker-first at all STC levels (`SOL_TAKER_FIRST=True`). Thin orderbook makes maker fills unreliable; the fee cost is worth paying for fill certainty.
- **BTC**: shorter escalation wait (7s vs. 15s default). BTC orderbook moves faster than other assets; a longer wait risks the spread moving away.
- **Decided contracts**: route direct taker regardless of STC. The whole point of a decided contract is high-conviction immediate execution — waiting for a maker fill defeats the purpose.

4.4.5 Amend over cancel-replace

Unfilled maker orders escalate via `amend_order()` (a single in-place price change) rather than cancel-then-replace. Amend preserves queue position; cancel-replace loses it. Empirically faster fills.

4.4.6 Fill detection

Primary: Kalshi WebSocket. Fills are reported in real-time at zero API cost. Backup: REST polling every 5s on outstanding orders. Both update `StateManager` immediately on confirmation.

4.4.7 Crash recovery

Every order's `order_id` is written to SQLite **before** the API submission. If the bot crashes between submission and confirmation, the next restart reconciles open positions from Kalshi's API against `StateManager` without duplicating orders or losing track of fills.

4.5 Conditional overlays

The bot has six live conditional overlays that augment the main 15M flow. Each is enabled by an independent gate and has its own per-asset configuration.

4.5.1 Decided contracts (DC)

DC identifies near-certain outcomes via extreme z-scores and routes direct-taker. Four live tiers as of May 2026:

Tier	Condition	Price range	Sizing
T1	$z \leq -5$	any	20% fixed

Tier	Condition	Price range	Sizing
T1B	$z \leq -4$	95¢+	20% fixed
T2	$z \leq -3$	93–96¢	20% fixed
T2-Z25	$z \leq -2.5$	93–96¢	10% fixed

T2-Z2 ($z \leq -2$) was returned to shadow on 2026-04-22 after underperformance (–\$313 across 47 trades). Six expansion shadows are collecting data for future tier promotion. Canonical spec: [kb/concepts/dc-strategy.md](#).

SOL-specific DC overrides: 5% sizing at ≥ 97 ¢, 10% sizing at 95–96¢ (tighter than the global DC scale due to SOL's thin orderbook).

4.5.2 Terminal momentum (TM)

Trades 96/98/99¢ contracts in the final 1–5 minutes. Sizing capped to 25–500 contracts. Near-100% standalone WR. Logic: in the closing minute, residual probability mass collapses sharply toward the realized outcome; high-price contracts with adequate cushion become near-deterministic.

4.5.3 Low-price near-expiry (LPNE)

BTC-only. Entry zone 80–87¢, STC 10–120s, 50 contracts fixed, gated by $\text{prob} \geq \text{price}/100$ (model conviction at the entry strike). Logic: in the final 2 minutes of a 15-minute window, mean-reversion candidates at sub-90¢ prices have well-defined edges if the model strongly supports the outcome.

4.5.4 Weekend discount

Sat/Sun, 90¢+ entries, STC ≤ 600 s, no DC overlap. Captures the empirical pattern that weekend liquidity is thinner — fewer market participants, less aggressive market-making — which leaves more contracts mispriced. The fallback Kelly-sign gate (added 2026-05-17 in the followup chain [7f4aeec](#) → [2113fba](#)) prevents the bug class where `_wknd_position == 0` ambiguously means “Kelly rounded to 0” vs. “Kelly clamped from negative” — the gate now fires only on `_wknd_kelly_f > 0`.

4.5.5 Overnight discount

Weekday 04–11 UTC, 89¢+, STC ≤ 600 s, no DC overlap. Same logic as weekend — overnight U.S. hours have thinner liquidity.

4.5.6 Loss-burst cooldown

Per-asset 2-hour lockout after any 15M loss on that asset. Counterfactual evaluation showed +\$441/30d expected savings. Logic: empirical loss clustering — a loss on one asset is statistically

followed by additional losses on the same asset within a short window, likely due to underlying market regime change that the model hasn't yet detected.

Part V: Multi-vertical Engines

5.1 S&P 500 intraday (KXINXU series)

The SPX engine trades 15-minute binary contracts on the S&P 500 index during NYSE regular trading hours (9:30 AM–4:00 PM ET). It uses the same EGARCH framework as crypto, adapted for equity-specific dynamics. Briefly promoted to live on 2026-03-17 and reverted same-day after Polygon.io returned 403 errors, breaking the primary price feed. Currently in observation mode.

5.1.1 Equity-adapted EGARCH

- **Leverage bounds:** $\gamma \in (-0.30, -0.05)$ — approximately 4× stronger than crypto. Reflects the empirically large equity leverage effect (down moves dominate up moves in volatility impact).
- **VIX integration:** when VIX-implied volatility diverges >30% from realized RK, the engine shifts 30% weight toward VIX. On startup, EGARCH is seeded from VIX to avoid a cold-start period.
- **Market hours guard:** NYSE RTH 9:30–16:00 ET with DST awareness and holiday calendar. Engine skips the first 10 minutes post-open (auction noise) and the last 5 minutes (auction tail).

5.1.2 Intraday seasonal deseasonalization

SPX volatility follows a documented U-shaped intraday pattern (high at open and close, low at mid-day). The engine deseasonalizes via 13 half-hour buckets (09:30–16:00 ET) with EWMA-calibrated seasonal factors, preventing systematic bias from time-of-day effects.

5.1.3 Configuration

Parameter	Value
Status	Observation only
Entry price range	90–99¢
STC window	300–1800s
Market blend	0/100 (CalEngine-only — no market blend)
Max risk per trade	10%

Parameter	Value
Kelly fraction	0.125 (eighth-Kelly)
Bankroll fraction	15% (SPX sizes off 15% of total balance)
Max positions per window	2
Max risk per window	15%
Fee multiplier (taker)	0.035 (half crypto)

5.1.4 HAR-RV shadow

A HAR-RV¹⁹ competitor runs in parallel as a shadow strategy. HAR-RV is a heterogeneous autoregressive realized-volatility model with three lagged components (1-day, 5-day, 22-day) — substantially simpler than EGARCH but with strong empirical performance for daily volatility forecasting.

5.2 Weather temperature (currently observation-only after kill)

The weather engine trades daily high-temperature prediction markets across 19 U.S. cities. The NO-side was LIVE in 1-contract verification mode from 2026-04-11 through 2026-05-16, then **killed**. Status as of May 2026: observation-only on both sides; re-research initiative active in folder 90149436180 on ClickUp.

5.2.1 Cities

19 cities with Kalshi series tickers KXHIGH{XX}: New York, Chicago, Miami, Denver, Los Angeles, Austin, Atlanta, San Francisco, Dallas, Phoenix, Philadelphia, Minneapolis, Seattle, Houston, Boston, Las Vegas, Oklahoma City, Washington DC, New Orleans.

5.2.2 Ensemble probability model

The engine queries Open-Meteo for two NWP ensemble systems:

Model	Members	Resolution	Provider
GFS Seamless	31	~13 km	NOAA
ECMWF IFS 0.25°	51	~25 km	ECMWF
HRRR (deterministic)	1	~3 km	NOAA

The 82 ensemble members (31 GFS + 51 ECMWF) yield a distribution of possible temperature outcomes. The engine fits a Gaussian (μ , σ) to the combined ensemble and computes:

¹⁹Corsi, F. (2009). “A Simple Approximate Long-Memory Model of Realized Volatility.” *Journal of Financial Econometrics* 7(2): 174–196.

- **Bracket markets:** $P(\text{lower} < T < \text{upper})$ via CDF difference
- **Threshold markets:** $P(T > \text{threshold})$ or $P(T < \text{threshold})$ via tail probability

5.2.3 Bias correction

Per-city EWMA bias tracker ($\lambda = 0.90$, 7-day half-life). After each day's actual temperature is observed:

$$\text{bias}_{\text{city}} = \lambda \cdot \text{bias}_{\text{prev}} + (1 - \lambda) \cdot (\text{actual} - \text{forecast})$$

The ensemble mean is shifted by this bias on subsequent forecasts.

5.2.4 Kill rationale (2026-05-16)

The NO-side LIVE phase from 2026-04-11 to 2026-05-16 generated $n = 167$ trades at the 39–40¢ NO entry zone with 38.3% WR (Wilson 95% CI [23.6%, 47.0%]). The prior assumed ~70% WR. The actual rate is materially below the prior and below the breakeven threshold for the fee-adjusted edge. RCA conclusion: the near-ATM zone (NO 39–40¢ YES 60–61¢) is the market-maker zone where institutional liquidity removes any model-based edge. The far-ITM NO band (4–12¢) showed 91.7% WR ($n=157$) in shadow bracket_no_live; the next iteration of weather research is anchored on that band.

5.3 Sports comebacks (observation, basketball alpha detected)

The sports engine monitors live games across 28 leagues for Bayesian comeback signals — situations where a pregame favorite is trailing in-game but statistically likely to recover.

5.3.1 Leagues

Binary outcome (home/away): NBA, NHL, MLB, NCAAB, NCAAF, NFL, WNBA, UFC, ATP Tennis, WTA Tennis. Esports (CS:GO, LoL, Valorant) — Kalshi price monitoring only.

Three-way outcome (home/draw/away): EPL, Bundesliga, La Liga, Serie A, UCL, Ligue 1, MLS, Liga MX, Europa League, Conference League, Super Lig, Eredivisie, World Cup, FIFA Friendlies, AFC Asian Cup.

5.3.2 Bayesian comeback model

Posterior comeback probability via lookup-table of empirically calibrated likelihood ratios:

$$P(\text{comeback} \mid \text{data}) = \frac{LR \cdot P(\text{prior})}{LR \cdot P(\text{prior}) + (1 - P(\text{prior}))}$$

The LR table is keyed on (deficit bucket, time remaining, pregame strength). Conservative scaling compresses LR 80% toward neutral to prevent overconfidence — necessary because the small per-game-state sample sizes of the underlying empirical calibration are noisy.

5.3.3 Per-sport CalEngines

Each sport group maintains its own CalEngine (sports_basketball, sports_baseball, sports_football, sports_soccer, sports_tennis, sports_hockey, sports_other). Per-sport calibration is essential because the LR transferability across sports is poor — basketball comebacks are structurally different from soccer comebacks.

5.3.4 Status

Basketball alpha detected: best-robust-filter is the NBA strong-config (pregame $\geq 60\%$, price $\leq 70\%$, time remaining $> 85\%$). 69.2% WR n=39 at last reading. SPRT (sequential probability ratio test) running — has not yet converged on stop/continue. Tennis is a drag (negative PnL). Per-sport-group CalEngines are learning in shadow. Promotion criteria: kb/decisions/sports-promotion-criteria.md. Refreshed 3× daily by bot/ai/researcher.py.

5.4 Hourly crypto (disabled)

Hourly cryptocurrency markets (KXBTCD, KXETHD, KXSOLD, KXXRPD, plus KXHYPED, KXDOGED, and KXBNBD which are in HOURLY_EXCLUDED_ASSETS/HOURLY_NO_EXCLUDED_ASSETS — KXBNBD added T1 2026-05-17 ticket 86b9zmj0c) with 75 strikes per event. **Currently disabled** (kill-switched 2026-04-18). Both HOURLY_LIVE_ENABLED=0 and HOURLY_NO_SIDE_LIVE=0 on the VPS.

5.4.1 Kill rationale

Briefly promoted to live trading in late February 2026; reverted after analysis showed correlated multi-strike exposure was producing concentrated losses. Specifically: when BTC moved sharply, the bot held positions across multiple strikes (e.g., 88¢/89¢/90¢/91¢) all with similar directional exposure. A single adverse move cleared all of them simultaneously.

5.4.2 Pre-kill data

BTC NO 40–54¢ had 53.9% WR (n=1,113, p=0.005). This was the only structurally profitable hourly cohort. ETH/SOL hourly are structurally unprofitable after fees at hourly timescales; XRP hourly is fundamentally broken (orderbook depth issues).

5.4.3 Re-enable path

Both env vars must flip to 1 on the VPS, plus restart. The dedicated hourly CalEngine has been disabled due to +44pp overconfidence; a temperature scaling approach ($T=1.45$) is used instead when re-enabled. Per-window position limits (max 2) and risk caps (15%) prevent the original correlated-loss failure mode.

Part VI: The Data Corpus

6.1 The thesis

The Data Corpus is structurally co-equal with the trading bot in this project. The investor whitepaper covers the strategic framing; this section covers the implementation.

The corpus is a medallion-architecture²⁰ data lake that began capturing on **2026-05-17**. Bronze (raw immutable WebSocket frames), silver (cleaned typed Parquet derivatives, planned), and gold (joined feature-ready tables, planned).

6.2 Bronze format

Each bronze line is a JSON object:

```
{"_wire_recv_ts":"2026-05-17T09:52:14.034501Z",
  "_source":"kalshi_ws",
  "_conn":"A",
  "_channel":"orderbook_delta",
  "_collector_seq":48201,
  "_raw":{"type":"orderbook_delta","msg":{"..."}}
```

The six-field envelope is the **minimum viable bronze schema**:

Field	Type	Why
<code>_wire_recv_ts</code>	ISO-8601 UTC microsecond	Captured AT INGRESS, pre-parse — cannot be reconstructed from <code>_raw</code> (server timestamp inside payload lacks receipt latency)

²⁰[Databricks Medallion Architecture](#). Bronze = raw immutable source of truth; Silver = cleaned, validated; Gold = business-ready aggregates.

Field	Type	Why
<code>_source</code>	string	kalshi_ws, coinbase_ws, open_meteo etc. — silver ETL dispatch key (the original D0.3 §1 slot reservation was nws_hrrr; D1.8 retracted that name because the bot polls Open-Meteo’s aggregated HRRR, not NWS direct)
<code>_conn</code>	string null	WS connection (A/B/C/D/E/F) for multi-conn Kalshi; null for REST snapshots. Lets silver QA detect single-conn outages without joining health logs
<code>_channel</code>	string null	orderbook_delta, trade, market_lifecycle_v2, or REST endpoint stub
<code>_collector_seq</code>	int	Monotone-increasing sequence number from collector boot — detects gaps independent of <code>_wire_recv_ts</code> . Resets on restart
<code>_raw</code>	string	The full unparsed wire payload. Bronze does NOT JSON-parse this field. Any future analysis that needs to verify “did Kalshi send X” has the verbatim payload

Irreversibility: the bronze schema cannot rev. If we later want a field not captured today, we cannot retroactively add it for past data. The 6 fields above are the minimum viable bronze envelope.

6.3 Partition scheme

Hive-style date-leading:

```
bronze/<source>/<channel>/year=YYYY/month=MM/day=DD/hour=HH/conn=<X>/<chunk_id>.jsonl.zst
```

Concrete example:

```
bronze/kalshi_ws/orderbook_delta/year=2026/month=05/day=17/hour=09/conn=A/20260517T095200Z_to_20260517T095659Z_seq4849340.jsonl.zst
```

Why date-leading: - Time-window queries dominate the access pattern. “Pull 14:30–15:00 UTC for backtest on 2026-05-20” hits one hour-prefix per source/channel/conn. - DEEP_ARCHIVE restore cost is per-object. A time-window query that nails a single hour prefix restores ~hundreds of MB; under category-leading, the same query might fan out to 30+ category prefixes. - Hive-style key=value partitioning is auto-detected by DuckDB, Athena, Spark, dbt-duckdb without configuration — silver ETL gets partition pushdown for free.

6.4 Rotation cadence

Rotate when **either** 5 minutes elapsed OR 100 MB uncompressed accumulated, whichever comes first.

- 5-min cap bounds data-loss window if the collector VPS crashes between rotation and upload.
- 100 MB cap targets the silver-Parquet-compaction sweet spot (50–200 MB rows after silver merge).
- Whichever-first because peak intervals (crypto vol spike) can drive 3–5× the byte rate.

Per-hour chunk count projection: off-peak ~50–150 chunks/hour across all 18 conn-channel combinations; peak 150–750 chunks/hour. Still well under S3 hot-shard limits (>5K req/sec on one prefix).

6.5 Upload contract — atomic-rename + rclone copy + verify + delete

The upload protocol implements crash-safe event-sourcing²¹ semantics:

```
def rotate_and_upload(in_flight_path: Path, source: str, channel: str, conn: str):
    chunk_id = f"{first_ts}_to_{last_ts}_seq{seq_start}-{seq_end}"
    s3_key =
    ↪ f"bronze/{source}/{channel}/year={yyyy}/month={mm}/day={dd}/hour={hh}/conn={conn}/{chunk_id}.jsonl.zst"

    fsync(in_flight_path)                                # 1. close + fsync
    ↪ in_flight
    tmp_path = TMP_DIR / f"{chunk_id}.jsonl.zst.tmp"
    zstd_compress(in_flight_path, tmp_path, level=6)      # 2. zstd compress to
    ↪ tmp/
    fsync(tmp_path)
    outbox_path = OUTBOX_DIR / f"{chunk_id}.jsonl.zst"
    os.replace(tmp_path, outbox_path)                    # 3. atomic rename to
    ↪ outbox/

    exit = rclone_copy(outbox_path, s3_key,               # 4. rclone copy (NOT
    ↪ sync)
                                flags=["--checksum", "--immutable"])
    if exit != 0:
        ALERT(f"rclone upload failed exit={exit}")
        return                                           # KEEP local – retry next
    ↪ tick
```

²¹Event Sourcing pattern (Azure Architecture Center); AWS Prescriptive Guidance.

```

if rclone_size(s3_key) != outbox_path.stat().st_size:           # 5. verify
    ↪ byte-exact size
    ALERT("size mismatch")
    return                                                       # KEEP local

outbox_path.unlink()                                           # 6. ONLY NOW delete local
in_flight_path.unlink()

```

Why each step:

- `rclone copy` not `sync` — `sync` mirror-deletes, would erase S3 objects when local files age out. Catastrophic-failure-mode-to-avoid.
- `--checksum` — bit-exact upload verification by content hash; survives clock skew.
- `--immutable` — exits with code 6 if a local file's content differs from a same-name S3 object. Tampering or bug-detection guard.
- `tmp/ → outbox/` atomic rename — POSIX `rename(2)` is atomic on same filesystem. `rclone` sees either the complete file or no file; never partial.
- **Never delete on `rclone` non-zero or size mismatch** — preserves the chunk for retry. Better to fill disk and alert than silently lose data.

6.6 S3 lifecycle policy

```

{
  "Rules": [
    {
      "ID": "bronze-archive",
      "Filter": {"Prefix": "bronze/"},
      "Status": "Enabled",
      "Transitions": [{"Days": 30, "StorageClass": "DEEP_ARCHIVE"}],
    },
    {
      "ID": "silver-archive",
      "Filter": {"Prefix": "silver/"},
      "Status": "Enabled",
      "Transitions": [{"Days": 90, "StorageClass": "GLACIER_IR"}]
    }
  ]
}

```

- Bronze: Standard → DEEP_ARCHIVE @ 30d (skip IA). Replay/backtest tolerates 12-hour DEEP restore; interactive analyst usage queries silver/gold instead. Skipping IA saves ~\$140/yr.
- Silver: Standard → GLACIER_IR @ 90d. Silver is regenerable from bronze, so we don't pay to keep hot forever.
- Gold: Standard, never expires, no transition. Hot tier for the bot and interactive operator queries.
- **Never expires** — every tier. Corpus value compounds with age; expiration is irreversible.

6.7 Collector isolation contract (load-bearing)

The collector runs in its own systemd unit with isolation knobs:

```

[Unit]
Description=Kalshi Data Corpus Collector
After=network-online.target
Wants=network-online.target

[Service]
Type=simple
User=botuser
WorkingDirectory=/home/botuser/kalshi-bot-repo
EnvironmentFile=/home/botuser/.env.collector
ExecStart=/home/botuser/kalshi-bot-repo/collector-start.sh
Restart=on-failure
RestartSec=10s

# CPUAffinity retired 2026-05-19 (ticket 86ba12rv6): the single-vCPU
# pin saturated at 90% CPU under 754K+ tickers; collector now floats
# both vCPUs with Nice=10 as the priority-isolation knob.
Nice=10                                # I/O-bound polite background; load-bearing isolation
MemoryHigh=400M                        # soft cgroup throttle at 78% (added 2026-05-20)
MemoryMax=512M                         # OOM the collector before it OOMs the box
MemorySwapMax=0
LimitNOFILE=4096                       # 6 WS conns + rotation + rclone fd headroom

StandardOutput=journal
StandardError=journal

[Install]
WantedBy=multi-user.target

```

Structurally:

Mechanism	What it guarantees
Separate systemd unit	OS-level process isolation
Separate Python process	Memory + GIL isolation
Separate WS conn(s)	Single-socket failure affects only that conn
Separate API key (KALSHI_COLLECTOR_KEY_ID)	Kalshi-side rate-limits/lockouts cannot cascade
import-linter forbidden contract collector → bot	Code-tree isolation enforced in CI
Separate state file (collector_state.db)	No SQLite write contention
Separate disk path (bronze_buffer/)	I/O isolation
Nice=10 (collector) vs Nice=0 (bot)	CPU contention bounded via priority — CPUAffinity retired 2026-05-19 ticket 86ba12rv6

Mechanism	What it guarantees
MemoryHigh=400M	Soft cgroup throttle before hard kill — surfaces memory pressure in cgroup counters at 78% of MemoryMax (added 2026-05-20 ticket 86ba12rf0)
MemoryMax=512M	RAM exhaustion of collector cannot OOM bot

The off-switch is `sudo systemctl stop kalshi-collector` — bot trading unaffected. Inverse off-switch: bot crash, collector keeps capturing.

The single shared failure surface is **disk full** (both processes write to the same root filesystem). Mitigations: rotate-then-delete-local discipline; D1.6 health-monitor alerts when disk is $\geq 80\%$ used (free space $< 20\%$); future option to put `bronze_buffer/` on a dedicated mount.

6.8 kalshi_wire/ — the shared transport layer

The original D0.3 design called for the collector to fully reimplement the Kalshi WebSocket protocol (zero bot.* imports). Mid-D1.2, an external advisor flagged this as wrong-sized:

“Capture and replay need to be two sides of the same coin for the data to serve your needs. Maybe it’s best to make a common capture module that can ingest crypto data for your current bot and pass the data to the decision engine, and then run a second instance of that module to save data for all the tickers you’re not currently trading. It’s gotta be the same architecture, infrastructure, module, set of rules.”

The amendment (2026-05-16, ticket 86b9zdhz2 D1.1.5, shipped f560d30) extracted a third top-level sibling package `kalshi_wire/` — peer of `bot/` and `collector/`. It houses:

- **RSA-PSS-SHA256 authentication** in `kalshi_wire/auth.py` (one canonical implementation; bot’s `bot/kalshi_client.py` previously had its own copy)
- **WebSocket client** in `kalshi_wire/ws_client.py` — `WSClient` handles connect, reconnect, silence-watchdog, message decode loop, and (when its `parse_on_demand` kwarg is `False` — the default for `bot/feeds/kalshi.py::KalshiFeed`) frame parsing into the 6-field envelope above. Post P1-B-brutalist Phase B1 (2026-05-20, ticket 86ba1qbf4), `collector/ws_connection.py::BronzeArchiver` constructs the wire with `parse_on_demand=True` to skip the per-frame `json.loads` on the `asyncio` thread (the dominant GIL-bound work at 705K-ticker universal mode); the collector recovers the routing fields it needs via substring-based ack detection + sid extraction.

Both `bot/feeds/kalshi.py::KalshiFeed` and `collector/ws_connection.py::BronzeArchiver` consume `kalshi_wire/`. The isolation contract is preserved by adding two new import-linter contracts: `kalshi_wire` → no `bot` and `kalshi_wire` → no `collector`. The library is structurally a pure-transport leaf.

A differential test at `tests/equivalence/test_kalshi_wire_differential.py` pins byte-equivalent frame capture between bot and collector — if either side’s parsing diverges, the test fails.

6.9 Day-zero and current state

Day-zero achieved 2026-05-17 ~09:52 UTC. Collector service running continuously since 09:57:59 UTC. First post-launch incident was a WebSocket 1009 “message too big” storm — Kalshi’s `type=subscribed/type=ok` acks include the cumulative ticker list per sid, which crosses the default `max_size=1` MiB of the websockets library at high subscription counts. Fix: raise `WSClient.ws_max_size` to 16 MiB (D1.3-fu1 PR #38 6b47153).

A subsequent 1011 “keepalive ping timeout” storm surfaced; the immediate stopgap (D1.3-fu3 `ping_timeout=30s`) reduced but did not eliminate the issue. Proper fix (D1.3-fu4 86b9zk4hz, in flight): decouple `BronzeArchiver._on_frame` writes to a worker thread so frame-receipt latency is independent of disk I/O latency.

Health monitoring shipped 2026-05-17 (D1.6 4dd7835): `scripts/ops/collector_health_monitor.py` cron alerts on disk pressure ($\geq 80\%$ used, i.e., free space $< 20\%$), WS reconnection storms (≥ 10 reconnects per 5-minute window, defaults), and service-down events. 10 contract tests.

6.10 Silver and gold — planned

Silver and gold layers are **regenerable from bronze**, so their construction is not on the critical path. The planned architecture:

- **Engine:** DuckDB + dbt-duckdb. Reads bronze directly from S3 via the `s3` extension; no meta-store required.
- **Compute target:** off-VPS. Default Mac M4 local; future option AWS Athena. Per the `feed-back_vps_compute_isolation` rule, silver/gold ETL must not contend with the bot for the 2-vCPU VPS.
- **State:** incremental models. `materialized='incremental'` re-runs only on new bronze partitions. Backfill is `dbt run --full-refresh --select <model>`.
- **Versioning:** silver and gold both path-versioned (`silver/v1/...`, `silver/v2/...`) AND row-versioned (`_silver_schema_version` column).

D2.1 (silver schema design) and D2.2 (ETL implementation) are upcoming tickets.

Part VII: The Agentic-Engineering Operating Model

7.1 Motivation

This project is operated by one person plus AI agents (primarily Claude via the Claude Code CLI). At time of writing, the platform has ~6,506 tests, multi-vertical engines, a corpus collector, automated

deploys, daily JSONL rotation, an analyst LLM, and a multi-Pillar quality discipline. The typical team-size estimate for this rate of output and quality discipline is 5–15 engineers; the actual headcount is one. This section describes how that arithmetic works in practice.

The published benchmark data is the empirical anchor for this thesis: as of May 2026, SWE-bench Verified shows that *the same LLM in different scaffolds* varies by 15+ percentage points.²² The model is roughly fixed; the discipline around it varies; the discipline is the moat.

7.2 The verification gap

The binding constraint on agentic engineering is what Karpathy²³ and Jason Wei²⁴ have called the verification gap (or asymmetry of verification): LLMs generate plausible code faster than humans can verify it, and the gap between “agent claims a change is done” and “we have evidence the change is correct” is the dominant source of agentic-engineering failure modes.

The five Pillars (described next) are an industrial response. Each Pillar closes one face of the verification gap.

7.3 The Pillars

Pillar 1 (foundational CI/test scaffolding — pytest tiers, GitHub Actions deploy pipeline, basic linting) is described in Part VIII (Testing Infrastructure) and Part IX (Infrastructure and Operations). Pillars 2–5 below are the agentic-engineering-specific extensions that close the verification gap on top of the Pillar 1 foundation.

7.3.1 Pillar 2 — Paved roads (hooks + skills)

Inspiration: Netflix Paved Road²⁵, Spotify Golden Path²⁶.

Make the desired path the easiest path. The agentic translation:

- **Claude Code hooks** intervene on tool invocations to block bad-shape commits. Examples: a hook prevents `git push --force` to main without explicit override; a hook blocks commits that introduce unused imports; a hook fires after every Edit tool call to verify the file still parses.
- **Skills** package well-tested workflows into invocable units. Examples: `/deploy` (push to main + verify VPS auto-deploy), `/audit` (run statistically-rigorous per-vertical audit with Wilson CIs), `/investigate` (emergency investigation of an anomaly), `/test-writer` (scaffold a failing regression test before any new code). The agent prefers skills to ad-hoc shell commands.

²²SWE-bench. For instance, Augment Code won February 2026’s identical-model bake-off by indexing the full repo first; MarkTechPost reports the scaffold-variance finding.

²³Karpathy on X (status 1930305209747812559); “Software 3.0” — Latent Space. Karpathy uses the painter/GAN analogy: “generation (1) and discrimination (2)... the faster the loop the better.”

²⁴Wei, J. “Asymmetry of Verification and Verifier’s Law.” jasonwei.net.

²⁵Netflix Paved Roads — Saif Rajhi.

²⁶Platform Engineering — Golden Paths That Actually Go Somewhere.

- **CLAUDE.md files** at every directory level auto-load context. The agent never needs to discover conventions by trial-and-error. `bot/CLAUDE.md` covers implementation rules; `tests/CLAUDE.md` covers test conventions; `scripts/CLAUDE.md` covers tooling; `ops/CLAUDE.md` covers infrastructure.

The agent reads, internalizes, and follows these conventions because they're loaded into context before it can act otherwise. Steinberger has written about the “just talk to it” school of agentic workflow²⁷ that *de-emphasizes* hooks; our approach is materially stricter than his. The argument for stricter: agents are clever, but agents also drift, and a hook fires deterministically while a context-loaded convention is only as reliable as the context window.

7.3.2 Pillar 3 — Equivalence snapshots

Engine outputs are pinned against a 1,000-row reference corpus. Test files in `tests/equivalence/` run the volatility engine, probability engine, and calibration pipeline against the corpus and assert byte-equivalent outputs against snapshots committed to git.

Snapshots are never auto-regenerated by agents. Regen is a human-with-diff-review operation; see `tests/equivalence/REGEN.md`. The reason: the entire purpose of the snapshot is to catch “agent confidently made a refactor that silently shifted model behavior.” Allowing the agent to regenerate the snapshot defeats that purpose.

This is the load-bearing protection against the most expensive failure mode in this project: a refactor that *looks correct*, *passes all tests*, and *silently re-trains the model on a contaminated distribution*. The R3 review of the Sprint A.1a `cal_mlp` refactor caught this exact regression — sigma winsorization landed in the train surface but not the serve surface; the equivalence snapshot detected the divergence.

7.3.3 Pillar 4 — TDD-first

The discipline: before any non-trivial change, an agent must write the failing regression test, confirm it fails on main, then make the smallest change that turns the test green.

The `/test-writer` skill scaffolds this: takes a description of the bug or new behavior, generates a failing test in the right location, runs it to confirm RED, and hands off to the agent for implementation.

The TDD-with-agents pattern is a Claude Code community / Anthropic engineering culture pattern²⁸, not a single canonical methodology. The discipline is what makes AI-generated code reviewable: the test asserts the change's *meaning*, and the human reviewer reads the test, not the implementation. If the test is wrong, the human catches it; if the implementation is wrong, the test catches it.

²⁷Steinberger — Optimal AI Dev Workflow; Just Talk To It.

²⁸InfoQ: Inside Claude Code Creator's Workflow; The New Stack: Claude Code and the Art of TDD; Pragmatic Engineer: How Claude Code is Built; alexop.dev: Forcing Claude Code to TDD.

7.3.4 Pillar 5 — Tiered test suite + testmon + mutmut

Tests are organized in 4 deploy-blocking in-tree tiers (plus the non-blocking research tier and out-of-band mutation testing):

Tier	Directory	Purpose
1	tests/contracts/	API contracts, DB signatures, public-interface invariants
2	tests/integration/	Cross-module flows, end-to-end scenarios
3	tests/unit/	Function-level pure logic
4	tests/equivalence/	Pinned engine outputs on 1000-row reference corpus
5 (research, non-deploy-blocking)	tests/research/	Falsification spike tests + cross-system research (paired with scripts/research/); may carry intentional NotImplementedError stubs during scaffold-first phase per TDD-first discipline. Excluded from deploy-blocking integration shards via INTEGRATION_IGNORES.
mutmut (out-of-band)	make test-mutmut	Mutation testing — runs against bot/engines/{volatility,probability}.py with the equivalence tier as runner; catches “test passes but doesn’t actually constrain”

testmon runs only the tests affected by current changes for fast feedback. The full suite takes minutes; testmon-filtered runs take seconds. CI runs the full suite; local development uses testmon for the inner loop.

mutmut (mutation testing) periodically mutates the codebase one operator at a time and checks whether any test still fails. A “live” mutant — a code change that doesn’t break a single test — indicates a gap in test coverage. The mutmut baseline is checked against make targets in CI.

Test count at time of writing: **~6,506 tests across ~270 test files organized in 4 deploy-blocking in-tree tiers (plus the non-blocking research tier and out-of-band mutation testing).**

7.3.5 Adversarial review (the distinctive pattern)

Risky changes (anything affecting the main loop, calibration pipeline, execution layer, or data corpus) are reviewed by a separate AI agent prompted to find every fault — pretending the change is being submitted by a hostile contributor.

The change ships only when the adversarial reviewer returns **two consecutive rounds with zero CRITICAL and zero MAJOR findings**. This is a production application of the Constitutional AI²⁹ and AI Safety via Debate³⁰ patterns: one agent generates, another critiques against an explicit rubric.

The adversarial reviewer is implemented as a Claude Code sub-agent (`.claude/agents/adv-reviewer.md`) with a specific prompt and tool restriction (read-only — Bash, Read, Grep, Glob; no Edit/Write). The prompt:

- Treat the change as if submitted by a hostile contributor.
- Find CRITICAL findings (would break production), MAJOR findings (correctness or maintainability problems), and minor findings (style, naming).
- Cite line numbers; do not propose fixes; do not be helpful.
- Return structured output with categorized issue lists.

In practice, R1 typically catches 3–8 issues, R2 catches 0–3 follow-up issues, and most “Bits” clear in 2–4 rounds. Particularly elaborate Bits with multi-surface lockstep concerns — D1.5 (collector systemd deploy), Path C (writer-IAM template), Bit 9.3-iii.c (`bot/_impl.py` deletion), D1.6 (collector health monitor), and others — have cleared in 7+ rounds; the canonical discipline anticipates this in its sacred-rule language (“some Bits need 8”).

A drift-sweeper sub-agent runs in parallel for changes that touch load-bearing names (file paths, function renames, constants, contracts) — it sweeps the tree for stale references and documentation drift, catching the sister-doc drift class that the narrower adversarial reviewer misses.

7.4 Import-linter contracts as architecture fitness functions

The codebase has 8 `import-linter`³¹ contracts in `.importlinter`, each enforcing a structural rule:

Contract	Forbids
<code>fetchers-no-engines</code>	<code>bot.fetchers.* importing bot.engines.*</code>
<code>feeds-no-engines</code>	<code>bot.feeds.* importing bot.engines.*</code>
<code>helpers-leaf</code>	<code>bot.helpers.* importing any sibling bot.X</code> subpackage (leaf-isolation rule — only <code>bot.constants</code> is permitted)

²⁹Bai, Y. et al. (2022). “Constitutional AI: Harmlessness from AI Feedback.” [arXiv:2212.08073](https://arxiv.org/abs/2212.08073).

³⁰Irving, G., Christiano, P. & Amodei, D. (2018). “AI Safety via Debate.” [arXiv:1805.00899](https://arxiv.org/abs/1805.00899).

³¹[import-linter on GitHub](#). The framing of import-linter contracts as architecture fitness functions follows Ford, Parsons & Kua, *Building Evolutionary Architectures* (O’Reilly, 2nd ed.). [Thoughtworks](#).

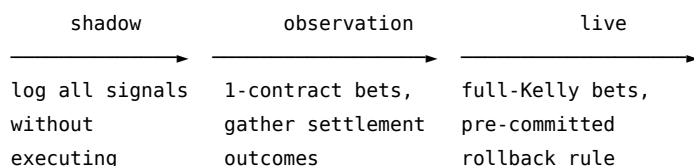
Contract	Forbids
bot-no-torch	bot.* (excluding cal_mlp serve paths) importing torch
bot-no-pandas	bot.* (excluding scripts/audit paths) importing pandas
collector-no-bot	collector.* importing bot.*
kalshi_wire-no-bot	kalshi_wire.* importing bot.*
kalshi_wire-no-collector	kalshi_wire.* importing collector.*

Each contract is an **atomic, continual, triggered architecture fitness function** in the Ford/Parsons/Kua sense. They run on every CI build via `make test-contract`. A contract violation fails the build before it can ship.

The contracts protect properties that human review cannot reliably enforce. “Collector should not import from bot” is structurally important — it’s the isolation guarantee that makes bronze immune to bot bugs — but it’s the kind of constraint a human reviewer can miss in a 200-line PR. The contract catches every violation, deterministically.

7.5 Shadow → observation → live promotion pipeline

Every new strategy or model change follows a structured promotion pipeline:



- **Shadow:** signal computed but not acted upon. `evaluated_opportunities` table accumulates rows with `filter_stage='shadow'` for analysis. No capital at risk. Used to characterize the strategy’s properties before any live exposure.
- **Observation:** 1-contract verification mode. Real exposure, real settlement outcomes, but at a size where worst-case is bounded. Used to validate that shadow numbers transfer to actual market conditions (slippage, fills, fees).
- **Live:** full Kelly-sized exposure. Promoted only with pre-committed rollback rule and a defined soak window. Examples: P2.3 HYPE/DOGE promoted with a 14-day Brier-monitored soak through 2026-05-28, per-asset Brier $\geq 5\%$ degradation → revert. P4.1 band-calibrated sizing has a 14-day band-stratified Brier soak through 2026-05-31, per-(asset × band) realized rate $\pm 5\text{pp}$ of baseline → revert.

Each promotion step has its own checklist in the relevant ticket (ClickUp), its own observability surface (Supabase dashboard panels showing live vs. shadow), and its own kill switch (`env var` or `OBSERVATION_MODE=True`).

7.6 The auto-research layer

bot/ai/ houses three LLM-driven research components:

- **bot/ai/analyst.py (Analyst)**: examines every losing trade post-settlement. Constructs a structured prompt with the trade context (entry price, exit price, model probability at entry, market price trajectory, settlement outcome) and asks Claude for a root-cause analysis. High-confidence findings (analyst-rated confidence ≥ 0.8) are routed to Telegram for operator notification.
- **bot/ai/researcher.py (researcher cron)**: runs three times daily. Refreshes shadow-system status, computes Wilson CIs on accumulated settlement data, runs SPRT updates on convergent strategies, and sends structured summaries via Telegram (split into multi-message chunks where needed). Referenced KB entries in kb/decisions/ (e.g. sports-promotion-criteria.md) are operator-maintained, not researcher-written.
- **bot/ai/auditor.py (audit-summary dispatcher)**: scheduled LLM-driven audit-summary generator that consumes daily JSONL journals + settlement state and writes operator-facing audit reports. Pairs with the /audit skill for on-demand invocation.

All three components consume the existing settlement data — none introduce a new data dependency. The architecture is: bot writes data, the analyst/researcher/auditor trio reads and synthesizes, operator reviews synthesis.

7.6.1 Future: closing the auto-research loop

The current auto-research is read-only (synthesis only). A future direction is closing the loop: the researcher proposes a new shadow strategy variant, the system instantiates it, settlement data accumulates, the researcher evaluates after a defined soak window, and if criteria are met the strategy enters the formal promotion pipeline.

This is bounded by the verification gap (§7.2) and the operator’s risk tolerance for autonomous changes to a live trading system. The pragmatic near-term path is:

1. Researcher proposes shadow variants (already happens via kb/decisions/ writes).
2. Operator manually approves and instantiates (currently a hand-edit of _strategy_registry).
3. The “manually approves” step gets a /propose-shadow skill that reduces it to a one-command approval.
4. The skill itself can run under a constrained automation rule (e.g., only auto-instantiate variants with sample-size below 1-contract verification thresholds).

Promotion to live remains a human decision indefinitely.

7.7 What the operating model enables

The pace of structurally significant change shipped in May 2026 alone is the empirical measure:

- Data Corpus bronze layer (5 PRs: #38, #41, #43, #46, #51 — 1009 ws_max_size raise (D1.3-fu1), deploy.yml pip install (D1.5.1), 1011 ping-timeout stopgap (D1.3-fu3), path-aware collector restart (D1.5.2), collector health monitor (D1.6))
- P2.3 HYPE/DOGE live promotion (atomic 3-commit deploy with per-asset constants + market blend weights)
- P4.1 band-calibrated sizing (42-cell hierarchical-shrunk lookup, 4 adv rounds)
- weekend_discount Kelly-sign followup chain (6 PRs closing an ambiguous-proxy class bug)
- CI performance Bits 2, 3, and 4.5 (pip cache, drop -v, session-scoped ast_cache fixture)
- Weather NO-side kill + re-research initiative scope

The traditional engineering team for this rate of change at this test discipline is 5–15 people. The actual headcount is one. The five-Pillar architecture is what makes the arithmetic work.

7.8 What the operating model does not solve

Honestly stated:

- **The model can't run while the operator sleeps.** The bot does — new development requires the operator. No “agent works overnight on a hard problem” yet; coding tasks remain interactive with frequent human checkpoints, particularly on high-risk changes.
- **Adversarial review is narrow.** It will find the specific bug class prompted; it will miss the bug class no one thought to ask about. Postmortems with named lessons (currently 100+ “L” lessons in kb/failures/ and kb/concepts/) are part of the cycle to grow the prompt rubric over time.
- **Attention budget is the binding constraint.** Adding more AI agents past a certain point does not produce more output; it produces more output the human has to verify. The relevant scaling is per-task quality (agent reliability + scaffold discipline), not per-agent count.
- **Some tasks remain human-only:** capital deployment, strategic kills, incident calls during live anomalies. Operational and judgment work do not delegate well.

Part VIII: Testing Infrastructure

8.1 Test tier overview

Tests are stratified in 4 deploy-blocking in-tree tiers (plus the non-blocking research tier and out-of-band mutation testing) (§7.3.4). Each tier has different runtime characteristics and different gating thresholds.

Tier	Files	Test count	CI gate
1 contracts	tests/contracts/test_1,438	1,438	Pre-commit + CI

Tier	Files	Test count	CI gate
2 integration	tests/integration/test_*.py	4,366	CI required
3 unit	tests/unit/test_*.py	245	CI required
4 equivalence	tests/equivalence/test_*.py (per-engine subdirs)	66	CI required
regression	tests/regression/ (failure-mode memory)	134	CI required
hooks	tests/hooks/ (Claude Code hook integration)	20	CI required
mutmut (out-of-band)	make test-mutmut against bot/engines/{volatility,probability}.py	ongoing	Periodic

Total in-tree: 6,506 tests at time of writing. Counts confirmed by `pytest --collect-only`.

8.2 Tier 1 — contracts

Contracts pin invariants that the rest of the codebase depends on. Examples:

- `tests/contracts/test_db_signatures.py` — every public method on `StateManager` has a frozen signature; signature changes require corresponding test changes.
- `tests/contracts/test_calmlp_lockstep.py` — AST guard that all 4 `cal_mlp` drift sites route through canonical helpers (`compute_derived_features`, `compute_hour_sin_cos`, `apply_sigma_winsor`).
- `tests/contracts/test_sprint_14_a_x5_watchdog_move.py` — locks the watchdog filesystem location post-Sprint-14-A move.

Contracts run in pre-commit hooks; CI re-runs them. The “shape” of the codebase is captured here.

8.3 Tier 2 — integration

End-to-end flow tests. Examples:

- `tests/integration/test_insert_schema_parity.py` — verifies that the DB insert statements in the bot match the actual schema.
- `tests/integration/test_watchdog_orphan_detection.py` — startup detects orphan PIDs holding the SQLite file.
- `tests/integration/test_orphan_db_watchdog.py` — end-to-end test of the orphan-DB watchdog cron.

8.4 Tier 3 — unit

Function-level pure-logic tests in `tests/unit/`. 245 tests at time of writing. Coverage focus: pure logic functions in scanner, executor, sizer, calibration, and edge-detection helpers. Most cross-module flow is covered at the integration tier (Tier 2, ~4,366 tests) — the unit tier intentionally stays small and fast.

8.5 Tier 4 — equivalence

`tests/equivalence/` pins engine outputs against a 1,000-row reference corpus. Snapshot files live in per-engine subdirectories (`tests/equivalence/test_volatility_engine/`, `tests/equivalence/test_probability_engine/`) following the `pytest-snapshot` convention. A snapshot test:

1. Loads the reference corpus (1,000 rows of historical market state).
2. Runs the engine (volatility, probability, calibration) on each row.
3. Compares the output byte-for-byte with the committed snapshot.
4. Fails on any divergence.

Snapshots are never auto-regenerated. Regen is a human-with-diff-review operation; the regen process:

1. Operator runs `pytest --force-regen tests/equivalence/`.
2. Git diff shows every changed snapshot.
3. Operator manually reads each diff, decides whether the change is intentional.
4. Operator commits the new snapshots with a justification in the commit message.

This is the load-bearing protection against silent refactor-induced regressions. The cost is operator attention on every snapshot change; the benefit is that every refactor either preserves byte-equivalent output or surfaces the deviation in code review.

The `test_kalshi_wire_differential.py` test is in this tier — it pins byte-equivalent frame capture between `bot/feeds/kalshi.py` and `collector/ws_connection.py`.

8.6 Out-of-band — mutation testing (mutmut)

`mutmut` mutates the codebase one operator at a time (e.g., $x > 0 \rightarrow x \geq 0$, $+$ $\rightarrow$ $-$) and checks whether any test still fails. A “live” mutant — a mutation that no test catches — indicates missing coverage.

Mutation testing is expensive (hours of runtime for the full codebase) and runs periodically rather than per-PR. Baseline `mutmut` output is committed to track regressions.

8.7 Test selection — testmon

testmon³² tracks which tests touch which production code paths. On subsequent runs, it only re-executes tests affected by current changes. The local development inner loop:

```
$ pytest                                # full suite, ~3 minutes
$ # edit bot/engines/volatility.py
$ pytest --testmon                       # only volatility-related tests, ~5 seconds
```

CI uses `make test-affected` which is a tighter testmon-driven selection. The full suite runs on the daily cron and on any change to test infrastructure itself.

8.8 The TDD-with-agents loop in practice

A concrete walkthrough — adding a new shadow strategy (“low-price-near-expiry-v2” or similar):

1. Operator invokes `/test-writer` “scaffold a test for the v2 LPNE strategy that should only fire at STC 5–60s for BTC at 80–84¢”.
2. The test-writer skill generates `tests/integration/test_lpne_v2.py` with a failing test that asserts the strategy fires under the specified conditions.
3. Skill runs `pytest tests/integration/test_lpne_v2.py` to confirm RED. Hands off.
4. Operator (or agent) implements the strategy in `bot/scanner/` and `bot/engines/`.
5. Operator runs `pytest --testmon` — test goes green, no other tests broken.
6. Adversarial review fires (R1, R2, ...) until 2-zero convergence.
7. Drift-sweeper checks sister documentation didn’t drift.
8. Commit, push, deploy.
9. Strategy starts as shadow; observation/promote pipeline takes over.

The cycle from “scaffold test” to “ship” is hours to days, not weeks.

8.9 Regression tests as failure-mode memory

Every bug that has caused a postmortem in `kb/failures/` has a corresponding regression test pinning the fix. Regression tests live in `tests/regression/` (134 tests at time of writing) or as named test methods within topical test files in `tests/integration/` and `tests/contracts/`.

Representative examples (verified to exist):

- `tests/integration/test_weekend_discount.py` — pins the L100 Kelly-sign ambiguous-proxy fix from 2026-05-17 and its sister-site Overnight anchor.
- `tests/contracts/test_drawdown_pct_hwm_source.py` + `tests/contracts/test_drawdown_scaler_reado` — pin the drawdown signal drift fix from 2026-05-15 (DD-2 / DD-3 / DD-4 atomic).
- `tests/integration/test_terminal_momentum.py` — pins the $|z| > 25$ threshold change.

³²[testmon docs](#).

The discipline: a bug fix never ships without a regression test that fails on the pre-fix code. The fix's correctness is asserted by the test; the test prevents the regression from re-occurring.

Part IX: Infrastructure and Operations

9.1 Production VPS

- Provider: DigitalOcean
- IP: 45.55.181.30
- OS: Ubuntu 24.04 LTS
- CPU: 2 vCPU
- RAM: 2 GB (no swap configured)
- User: botuser
- Hostname: configured per the operational runbook

Why this size: the bot's compute footprint is dominated by network I/O and small calculations. The 2-GB RAM ceiling is the binding resource constraint (Python + numpy + the cal_mlp inference at ~200 MB resident, plus the various WS buffers and SQLite cache). Larger sizes are available; this size is structurally sufficient and explicitly avoided on-VPS heavy compute (silver/gold ETL, cal_mlp training) per the feedback_vps_compute_isolation rule.

9.2 Systemd units

```
# ops/kalshi-bot.service (verbatim from repo)
[Unit]
Description=Kalshi Trading Bot
After=network-online.target
Wants=network-online.target

[Service]
Type=simple
User=botuser
WorkingDirectory=/home/botuser/kalshi-bot-repo
EnvironmentFile=/home/botuser/kalshi-bot-repo/.env
ExecStart=/home/botuser/kalshi-bot-repo/start.sh
Restart=always
RestartSec=30
StandardOutput=journal
StandardError=journal

[Install]
WantedBy=multi-user.target
```

The collector unit is in §6.7. Both units are managed via `ops/install.sh` — a multi-unit installer with a two-pass design (validate-all then install-all) that prevents half-installed state.

9.3 Deploy pipeline

GitHub Actions on push to main:

1. SSH into VPS as botuser.
2. `cd ~/kalshi-bot-repo && git fetch origin main && git checkout main.`
3. **Pre-deploy systemd unit drift check** (Bit 2.0.5.2): `diff <(systemctl cat kalshi-bot) <(git show ${github.sha}:ops/kalshi-bot.service)`. If the on-VPS unit has drifted from the unit at the deploy commit, abort the deploy before any `reset --hard`. The recovery path on drift is documented in the failure message.
4. `git reset --hard ${github.sha}` — pin to the exact deploy SHA, not origin/main (which can race against parallel auto-commits like the [skip ci] whitepaper PDF push).
5. `~/kalshi-bot-repo/venv/bin/pip install -r requirements.txt` (D1.5.1 — added 2026-05-17 after a zstandard dependency drift incident).
6. Unconditional bot restart: `sudo systemctl restart kalshi-bot`.
7. Path-aware collector restart (D1.5.2 — added 2026-05-17): if the diff touches `collector/`, `kalshi_wire/`, `ops/kalshi-collector.service`, `collector-start.sh`, or `requirements.txt`, AND `kalshi-collector` is currently active, run `sudo -n /bin/systemctl restart kalshi-collector`. If the operator has stopped the collector, the path-aware step skips to preserve operator intent.
8. Verify `systemctl is-active` on the restarted services.

Auto-deploy means a push to main is a production deploy. The discipline is to **never push to main without operator approval per the CLAUDE.md no-deploy-without-confirm rule**. Pushes to feature branches are safe.

9.4 SQLite and journals

`state.db` is the bot's primary state store. Key properties:

- **WAL mode** (Write-Ahead Logging) for crash resilience.
- **busy_timeout=30000** — 30 second timeout on lock contention.
- **3-retry-on-busy** wrapper at the StateManager level — retry small commits if WAL blocking is transient.
- **≤50-row commit batches** — bounds the size of any single write and reduces lock contention.
- **recent_writes ring buffer + slow-batch breakdown logging** — when a commit takes >100ms, the breakdown is logged for forensic analysis.

JSONL journals (`logs/journals/`) record every event (scans, opportunities, rejections, trades, settlements, maker fill model training data). Append-only. Daily rotation + zstd compression via `rotate_journals.sh` cron at 04:00 UTC (script lives on the VPS, not in git). The compressed files

upload to S3 at 04:30 UTC via `journal_archives_s3_sync.py` (same `--checksum --immutable` relone discipline as the data corpus).

Retention: 90 days local on VPS; S3 lifecycle transitions to DEEP_ARCHIVE at 30 days, never expires.

9.5 Watchdogs and alerts

- **Telegram notifier:** trade alerts, large fills, errors, Analyst high-confidence findings. Per-event de-duplication to prevent alert storms.
- **Supabase dashboard:** real-time web UI (30s sync interval) showing positions, P&L, volatility per asset, orderbooks, execution health, calibration diagnostics, shadow-system data. Hosted on gh-pages with Supabase Realtime as the backend.
- **Collector health monitor** (`scripts/ops/collector_health_monitor.py`, cron): disk pressure, WS reconnection storms, service-down. Telegram via `bot.notifier.TelegramNotifier`.
- **Orphan-DB watchdog** (`bot/orphan_db_watchdog.py` at startup + `ops/watchdog.py` periodic cron): detects PIDs holding the SQLite file when no bot process should be running. Three-layer prevention since 2026-05-03 (wrapper signal escalation in `scripts/h4_run_with_alert.py` + script SIGALRM 25-min + startup `lsof` watchdog).
- **15M silence watchdog:** alerts if no productive 15M scan tick observed in 10+ minutes (with a 15-minute min-uptime guard to suppress startup false positives). Catches main-loop hangs that systemd doesn't (the process is running but the inner loop is stalled).

9.6 Crash recovery

Order identifiers are written to SQLite **before** API submission. On crash, the next startup reconciles state with Kalshi's API on a strict API-wins basis:

1. Query Kalshi REST for all currently-resting orders (`client.get_orders(status="resting")`).
2. For every resting order found, **cancel it on Kalshi** (`client.cancel_order(oid)`) — startup clean-slate discipline. Stale orders from a crashed run never persist; the `STALE_ORDER_CLEANUP` log marker is the load-bearing signature.
3. Sync local position state from Kalshi's positions API — API is the source of truth, the local `state.db` is updated to match.
4. Local `pending_orders` rows whose `status='resting'` are reconciled to `status='canceled'` to match the Kalshi-side action.

The discipline is “API wins” — the bot's `state.db` never overrides Kalshi's record of what's open and what isn't. This handles the in-flight-at-crash case without duplicate orders or stale phantom positions.

9.7 Auto-research crons

Cron	Frequency	What it does
<code>scripts/audit/calibrator_feature_health.py</code>	Every 6h	Checks every <code>cal_mlp</code> feature for >99% population (95% for known-flaky 5min momentum). Telegram alert on <code>SCHEMA_DRIFT</code> .
<code>bot/ai/researcher.py</code>	3× daily	Refreshes shadow-system Wilson CIs, SPRT updates; sends summaries via Telegram.
<code>rotate_journals.sh</code> (lives on VPS, not in git)	Daily 04:00 UTC	Rotates JSONL journals and compresses with <code>zstd</code> .
<code>journal_archives_s3_sync.py</code>	Daily 04:30 UTC	Uploads <code>zstd</code> -compressed rotated journals to S3.
<code>state_db_s3_backup.py</code>	Daily 06:00 UTC	Backs up <code>state.db</code> to S3 (daily/ prefix).
<code>export_market_obs_to_s3.py</code>	Daily 05:30 UTC	Archives <code>market_observations</code> table to S3 (market_obs/ prefix).
<code>collector_health_monitor.py</code>	Cron (configured per VPS — installed by operator)	§6.9.

9.8 The skill catalog (operator-facing)

`/status` — 30-second pulse check (is data flowing, any trades, anything notable). `/audit` — statistically rigorous numbers w/ Wilson CIs for a specific vertical. `/investigate` — emergency investigation of an anomaly, alert, suspected bug. `/deploy` — push to main + verify VPS auto-deploy succeeded. `/data-health` — instrumentation quality check (NULL rates, data gaps, shadow coverage). `/alpha-audit` — cross-system “where’s the alpha” funnel. `/shadow` — all 5 shadow systems at a glance. `/variant-status` — specific variants (A1/A2, hourly alts) with Kelly-sized sim PnL. `/15m-alpha`, `/hourly-alpha`, `/spx-alpha`, `/weather-alpha`, `/sports-alpha` — per-system deep dives. `/maker-cost` — maker vs. taker opportunity cost. `/no-side` — NO-side data report. `/weekend-discount` — weekend/overnight discount status. `/research-package` — compile self-contained data package for external researcher. `/test-writer` — scaffold a failing TDD test before bot/ extraction. `/kb-lint`, `/kb-ingest`, `/kb-evolve` — KB health and maintenance.

Each skill lives under `.claude/skills/<name>/SKILL.md` and is invocable by typing the slash command in Claude Code. Skills compose well: `/investigate` often invokes `/audit` mid-investigation; `/deploy` invokes a `verify-step` internally.

Part X: Roadmap and Future Plans

10.1 Near-term (next 30 days)

- **D1.3-fu4 worker-thread decouple:** the 1011 WebSocket keepalive timeout storm needs a proper fix. Stopgap (`ping_timeout=30s`) is insufficient under steady-state load. The proper fix decouples `BronzeArchiver._on_frame` writes to a worker thread so frame receipt is independent of disk I/O latency.
- **P4.1 soak completion:** 14d band-stratified Brier soak ends 2026-05-31. Per-cell rollback rule fires on any (asset × band) drift $\geq 5pp$.
- **HYPE/DOGE 14d soak completion:** ends 2026-05-28. Per-asset Brier rollback rule.
- **D1.7 bronze lifecycle hygiene:** verify DEEP_ARCHIVE transition at +30d. Cost-validation report on per-object byte distribution + DEEP_ARCHIVE bill-vs-bytes ratio for sparse streams.
- **Weather re-research:** per the May 16 kill, the far-ITM NO band (4–12¢) showed 91.7% WR (n=157) in shadow. Re-research initiative anchors on this band; new shadow scope being scoped in folder 90149436180.

10.2 Cal_mlp v2 and v3

Two cohort upgrades are in the pipeline:

- **v2** (+8 features beyond v1.1): momentum features (1m/5m), buffer features (distance to next strike), BTC RV (cross-asset volatility). K=1 train target 2026-05-19.
- **v3** (+3 features beyond v2): spread, flow, CB-Kraken gap. K=2 train target 2026-06-22.
- **External market data integration:** OKX funding+OI for all 7 assets, Deribit BTC/ETH DVOL. Earliest v3 use 2026-06-22. The `external_market_data` table does not exist on VPS (the poller `scripts/backfill/external_market_poller.py` is CRON-NEVER-INSTALLED — open ticket to install daily cron and create the table on first run).

Each cohort upgrade ships atomically: train surface, serve surface, sigma winsorize, `cfg_fp` bump, lock-step contract test all in one commit. The Sprint A.1a / A.1b discipline (canonical helper homes, drift-site enumeration, AST guard) ensures the surface remains in lock-step across cohort upgrades.

10.3 Silver and gold

The bronze layer is the load-bearing piece; silver and gold are regenerable. Planned:

- **D2.1 silver schema design:** column-typed Parquet schemas for `kalshi_orderbook_nbbo`, `kalshi_trades`, `market_lifecycle_v2`, with path-versioning (`silver/v1/...`).
- **D2.2 silver ETL:** DuckDB + dbt-duckdb implementation. Off-VPS (Mac M4 local; future option AWS Athena). Incremental models; `materialized='incremental'` re-runs only new bronze partitions.
- **D2.3 silver QA:** gap detection by `_collector_seq`; conn-level outage detection; schema-drift dispatch.

- **D3.x gold:** joined feature tables for direct bot consumption (`settlement_labels`, `spot_at_decision`, `nbbo_at_decision`).

Timeline: not on critical path. Silver/gold are research-tier infrastructure that can be built once bronze accumulates enough data to be useful — likely Q3 2026.

10.4 SPX re-promotion

SPX has been observation-only since the 2026-03-17 same-day revert. Re-promotion criteria:

1. Polygon.io 403 resolved or alternative primary feed validated for ≥ 30 days continuous uptime.
2. Per-strategy Brier improvement on SPX-D CalEngine against passthrough on ≥ 500 settled outcomes.
3. Eighth-Kelly soak (1-contract verification) for ≥ 21 days.
4. Pre-committed rollback rule on per-window correlated losses (the original kill rationale).

No estimated re-promotion date — gated on Polygon stability.

10.5 Hourly re-enablement

Hourly is disabled since 2026-04-18. Re-enablement requires:

1. Both `HOURLY_LIVE_ENABLED=1` and `HOURLY_NO_SIDE_LIVE=1` env vars flipped.
2. Per-window position limit (max 2) and per-window risk cap (15%) verified in CI.
3. Temperature scaling $T=1.45$ applied to hourly CalEngine.
4. BTC-only initial scope (ETH/SOL hourly are structurally unprofitable after fees; XRP hourly is fundamentally broken).
5. NO 40–54¢ band first (only structurally profitable cohort pre-kill, 53.9% WR $n=1,113$ $p=0.005$).

No estimated re-enable date.

10.6 Closing the auto-research loop

Currently the researcher is read-only synthesis. Closing the loop:

1. **Researcher** → **propose**: writes structured proposals to `kb/decisions/` with explicit “would-fire-on” criteria.
2. **Operator** → **approve**: `/propose-shadow` skill reduces the manual instantiation to one command.
3. **System** → **instantiate**: shadow strategy entered in `_strategy_registry` automatically.
4. **Soak** → **observation**: defined window with measurement.
5. **Researcher** → **evaluate**: post-soak Brier / SPRT analysis with stop-or-continue recommendation.
6. **Operator** → **promote**: still a human decision (verification gap reasons).

The end-state is operator-as-gatekeeper-only: agent proposes, system runs the experiment, agent evaluates, operator approves promotion to live. Step 6 remains human indefinitely.

10.7 Selective hire trajectory

The operating model in §7 scales further than initially obvious but has a ceiling. Selective hire trajectory in priority order:

1. **Part-time quant researcher** for ongoing model R&D (cal_mlp v2/v3, novel calibration approaches, microstructure research using the corpus). 10–20 hrs/week contract.
2. **Operations contractor** for weeks-during-operator-vacation continuity. Documented run-books; no novel decisions delegated.
3. **Second engineer** once the codebase warrants it. Probably ~12 months out at current change velocity.

No hiring is planned in the next 90 days.

10.8 Open problems

Known limitations that remain unresolved at time of writing:

- **Cursor race rare tick error:** another row available SQLite error ~1 in 45 min. Shape D fix attempt 2026-05-03 introduced 460 lock errors in 2 min and was reverted. Real baseline post-orphan-kill is ~0/30 min. Hardware upgrade (\$18 → \$48 droplet) deferred.
- **Settlement watermark race:** stuck positions + stale orders. Fix written, pending deploy.
- **Drawdown signal real source:** PositionSizer.get_rolling_hwm() vs. session-HWM-in-memory-only. DD-6 URGENT operator decision to accept reset (Option B) — in-memory-only HWM resets at every bot restart, creating a discontinuity. Filed as open issue.
- **External market data writer:** external_market_data table does NOT exist on VPS; external_market_poller.py is operationally dormant. Open ticket to install daily cron.
- **HYPE/DOGE replay corpus:** P2.3.a remains blocked on 86b9xednb — HYPE/DOGE replay corpus proxy.

10.8.1 Recently closed (historical context)

- **Variant A (MAKER sub-floor fill):** FIXED Apr 22 commit 4bee926 (–\$582/30d). Variant C (1¢ thin-clamp): FIXED Apr 24 commit 4d7065a. Variant B (TAKER IOC sub-floor): decided not-to-fix (cohort net +\$178/22d).
- **Orphan-DB 3-Layer Prevention** (May 3 incident class): structurally closed by 59e7e84 wrapper signal escalation + 14225ff script SIGALRM 25-min + ccead4c startup lsof watchdog.
- **database is locked FAST-fail:** mitigated 2026-05-09 via cf34b5c 3-retry-on-busy + slow-batch breakdown shipped atop 2e56bff recent_writes ring buffer.

10.9 The corpus-as-substrate vision

The long-term vision is: the corpus is the substrate, the bot is one consumer, future bots are other consumers, and the data appreciation underlies all of it.

A concrete plausible future state, 3 years out:

- 3 years of byte-exact Kalshi WebSocket frames in S3 (~30–50 GB compressed/year × 3 years).
- Multiple silver/gold derivative tables in versioned paths, regenerated as schema-needs evolve.
- A research interface for academic researchers studying Kalshi microstructure.
- 2–3 production trading strategies trained on the corpus, each substantially more sophisticated than today’s (current strategies are calibrated against months of data; future strategies will train against years).
- A separate corpus for adjacent asset classes (Coinbase/Kraken crypto WS at byte fidelity; planned D1.9 future).
- Selective external partnerships where research interest aligns with our research interest.

The financial value of this state is hard to bound precisely. The lower-bound case is “internal research substrate that makes every successive bot version better.” The upper bound is “data-as-asset business in the style of Bloomberg or Quandl-pre-Nasdaq.” We are positioned for the lower bound by construction; the upper bound is optionality.

Part XI: References

The following are the primary sources cited by inline footnote elsewhere in this document. Page numbers, DOIs, and venue identifiers reflect the published / canonical form where verified.

Academic — volatility, calibration, distributions

- Barndorff-Nielsen, O. E. (1997). “Normal Inverse Gaussian Distributions and Stochastic Volatility Modelling.” *Scandinavian Journal of Statistics* 24(1): 1–13.
- Barndorff-Nielsen, O. E., Hansen, P. R., Lunde, A., & Shephard, N. (2008). “Designing Realized Kernels to Measure the ex post Variation of Equity Prices in the Presence of Noise.” *Econometrica* 76(6): 1481–1536.
- Barndorff-Nielsen, O. E., & Shephard, N. (2006). “Econometrics of Testing for Jumps in Financial Economics Using Bipower Variation.” *Journal of Financial Econometrics* 4(1): 1–30.
- Bates, J. M., & Granger, C. W. J. (1969). “The Combination of Forecasts.” *Operational Research Quarterly* 20(4): 451–468.
- Corsi, F. (2009). “A Simple Approximate Long-Memory Model of Realized Volatility.” *Journal of Financial Econometrics* 7(2): 174–196.
- Kull, M., Silva Filho, T., & Flach, P. (2017). “Beta calibration: a well-founded and easily implemented improvement on logistic calibration for binary classifiers.” *AISTATS 2017*, PMLR 54:623–631.

- Lee, S. S., & Mykland, P. A. (2008). “Jumps in Financial Markets: A New Nonparametric Test and Jump Dynamics.” *Review of Financial Studies* 21(6): 2535–2563.
- Mincer, J. A., & Zarnowitz, V. (1969). “The Evaluation of Economic Forecasts.” In *Economic Forecasts and Expectations*, NBER / Columbia University Press, pp. 3–46.
- Nelson, D. B. (1991). “Conditional Heteroskedasticity in Asset Returns: A New Approach.” *Econometrica* 59(2): 347–370.
- Niculescu-Mizil, A., & Caruana, R. (2005). “Predicting Good Probabilities with Supervised Learning.” *ICML 2005*.
- Platt, J. C. (1999). “Probabilistic Outputs for Support Vector Machines and Comparisons to Regularized Likelihood Methods.” *Advances in Large Margin Classifiers*, MIT Press, pp. 61–74.
- Timmermann, A. (2006). “Forecast Combinations.” *Handbook of Economic Forecasting* Vol. 1, Ch. 4, pp. 135–196.
- Vovk, V. (2012). “Conditional validity of inductive conformal predictors.” *PMLR* 25.
- Vovk, V., Gammerman, A., & Shafer, G. (2005). *Algorithmic Learning in a Random World*. Springer.

Academic — Kelly, drawdown, position sizing

- Goetzmann, W. N., Ingersoll, J. E., & Ross, S. A. (2003). “High-Water Marks and Hedge Fund Management Contracts.” *Journal of Finance* 58(4): 1685–1718.
- Grossman, S. J., & Zhou, Z. (1993). “Optimal Investment Strategies for Controlling Drawdowns.” *Mathematical Finance* 3(3): 241–276.
- Kelly, J. L., Jr. (1956). “A New Interpretation of Information Rate.” *Bell System Technical Journal* 35(4): 917–926.
- MacLean, L. C., Thorp, E. O., & Ziemba, W. T. (eds.) (2011). *The Kelly Capital Growth Investment Criterion: Theory and Practice*. World Scientific.
- Thorp, E. O. (2006). “The Kelly Criterion in Blackjack, Sports Betting, and the Stock Market.” *Handbook of Asset and Liability Management* Vol. 1, pp. 385–428.

Industrial — agentic engineering, software architecture

- Bai, Y. et al. (2022). “Constitutional AI: Harmlessness from AI Feedback.” [arXiv:2212.08073](https://arxiv.org/abs/2212.08073).
- Ford, N., Parsons, R. & Kua, P. (2023). *Building Evolutionary Architectures: Automated Software Governance* (2nd ed.). O’Reilly.
- Irving, G., Christiano, P. & Amodei, D. (2018). “AI Safety via Debate.” [arXiv:1805.00899](https://arxiv.org/abs/1805.00899).
- Karpathy, A. (2025). “Software 3.0.” [Latent Space](https://latent.space/).
- Wei, J. “Asymmetry of Verification and Verifier’s Law.” jasonwei.net.
- Wunderlich, F., & Memmert, D. (2023). “Machine learning for sports betting: should forecasting models be optimised for accuracy or calibration?” [arXiv:2303.06021](https://arxiv.org/abs/2303.06021).

Industrial — data architecture

- [Databricks Medallion Architecture](#).
- [Event Sourcing pattern \(Azure Architecture Center\)](#).

Operational — Kalshi documentation

- [Kalshi API Documentation](#).
- [Kalshi Historical Data API](#).
- [Kalshi Fee Schedule](#).
- [Kalshi Market Integrity / Regulation](#).

Background reading (not cited inline in this technical paper; listed for the interested reader)

- Berg, J., Nelson, F. & Rietz, T. (2019). “Longshots, overconfidence and efficiency on the Iowa Electronic Markets.” *International Journal of Forecasting* 35(1).
- Brier, G. W. (1950). “Verification of Forecasts Expressed in Terms of Probability.” *Monthly Weather Review* 78(1): 1–3.
- Cont, R., Stoikov, S., & Talreja, R. (2010). “A Stochastic Model for Order Book Dynamics.” *Operations Research* 58(3): 549–563.
- DeGroot, M. H. & Fienberg, S. E. (1983). “The Comparison and Evaluation of Forecasters.” *The Statistician* 32(1-2): 12–22.
- Gould, M., & Bonart, J. (2016). “Queue Imbalance as a One-Tick-Ahead Price Predictor in a Limit Order Book.” *Market Microstructure and Liquidity* 2(2).
- Hasbrouck, J. (1995). “One Security, Many Markets: Determining the Contributions to Price Discovery.” *Journal of Finance* 50(4): 1175–1199.
- Helmer, H. (2016). *7 Powers: The Foundations of Business Strategy*.
- Manski, C. (2006). “Interpreting the predictions of prediction markets.” *Economics Letters* 91(3): 425–429.
- Noonan, A., & Smith, P. (2024). “Application of the Kelly Criterion to Prediction Markets.” [arXiv:2412.14144](#).
- Page, L. & Clemen, R. T. (2013). “Do Prediction Markets Produce Well-Calibrated Probability Forecasts?” *Economic Journal* 123(568): 491–513.
- Shafer, G. & Vovk, V. (2008). “A Tutorial on Conformal Prediction.” *JMLR* 9: 371–421.
- Snowberg, E. & Wolfers, J. (2010). “Explaining the Favorite-Longshot Bias.” *NBER Working Paper* 15923.
- Thaler, R. & Ziemba, W. (1988). “Anomalies: Parimutuel Betting Markets.” *Journal of Economic Perspectives* 2(2): 161–174.
- Whelan, K. (2025). “Makers and Takers: The Economics of the Kalshi Prediction Market” (working paper). [karlwhelan.com](#).
- Wolfers, J., & Zitzewitz, E. (2004). “Prediction Markets.” *Journal of Economic Perspectives* 18(2): 107–126.

Document last updated: 2026-06-11T21:26:53Z